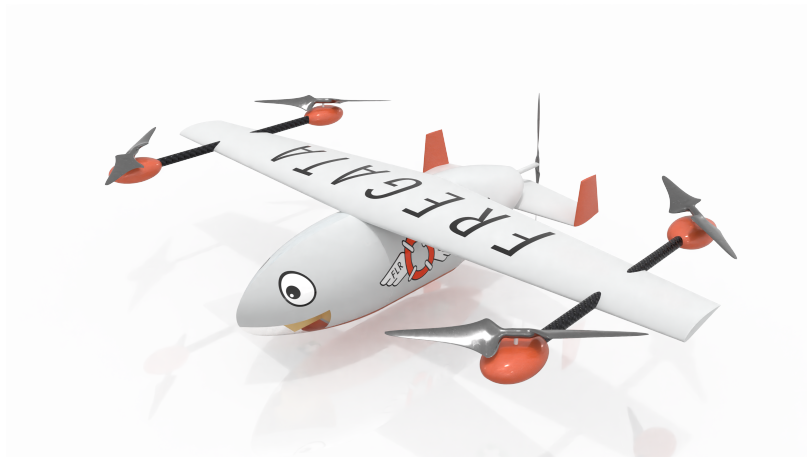




Flying Lifter

Performance Estimation



Kristoffer Ahlner
CID: 01715633

Group Design Project
H401 MEng Aeronautical Engineering

20/06/2022

FLR22	
Subteam	Members
Technical Directors	Foster Ng
	Julian Fung
	Daniel Earl
Aerodynamics	Anthony Haiser
	Kwok Leung Yim
	Marius Borsu Lefevre
Structures	Sorawis Itthithavorn
	Antoine Rieuneau
	Olivier Sussfeld
Propulsion and Systems	Jordi Frittoli
	Matan Hamburger
	Wei Han Lee
Performance, Stability & Control	Natalia Melo Fernandez
	Farsai Sittichoksuwan
	Kristoff Ahlner
Simulation	Joshua Binns
	Elia Chimonides
	Luis Ferreira da Silva Marques
Simulation	Yi En Puah
	Bojun Vito Yang
	Gabisanth Seyon
Simulation	Rohit Singh

Abstract

This report investigates the performance estimation of the Flying Liferaft solution. It was found that the UAS matched the majority of constraints stipulated in the design brief, however multiple design criteria were not met. It was found that the design climb rate of 500 fpm was achievable, with a maximum calculated climb rate of 861 fpm. The cruise speed was found to be marginally achievable at sea level conditions only. An optimisation study was conducted to investigate the most efficient method of transitioning to level flight, with the most efficient method being a level transition consuming 224 kJ. The overall minimum battery mass required to satisfy the energy requirement amounted to 1.49 kg, which is less than minimum mass to satisfy the voltage and current constraints therefore giving the UAS excess capability. The total estimated fuel required came to 2.94 kg, 0.983 kg more than expected in initial sizing. Improvements to the UAS would be to improve the specific excess power through either increasing the power of the propulsion unit or, more significantly, reducing C_{D0} .

Contents

List of Figures	iv
List of Tables	v
1 Introduction	1
2 Point Performance	1
2.1 Climb Performance	1
2.2 Turn Performance	2
3 Transition Optimisation	2
3.1 Design Process	3
3.2 Launch Method	4
3.2.1 Launch Method 1	4
3.2.2 Launch Method 2	5
3.2.3 Comparison	6
3.3 Transition Methods	6
3.3.1 Transition Method A	7
3.3.2 Transition Method B	7
3.3.3 Transition Method C	8
3.3.4 Transition Method D	8
3.4 Further Effects	9
3.4.1 Ground Effect	9
3.4.2 Effective Translational Lift and Transverse Flow	9
3.4.3 Stall Speed	9
4 VTOL Performance	10
4.1 Minimum Hover Power Requirement	10
4.2 VTOL Maximum Ceiling	10
5 Mission Performance	10
5.1 Assumptions in Mission Performance	10
5.1.1 VTOL Takeoff and Transition	10
5.1.2 Climb, Cruise and Descent to Loiter	10
5.1.3 Loiter and Descent to drop altitude	11
5.1.4 Dropping the Payload and VTOL Landing	11
5.2 Summary of Mission Performance	11
6 Conclusion	12
References	13
Appendix	14

List of Figures

1	Specific Excess Power Plot	1
2	Maximum Turning Load Factor	1
3	Methods investigated in the Transition Optimisation	3
4	Launch Method Force Diagrams	4
5	Climb Method 1	5
6	Climb Method 2	6
7	Transition Method Force Diagrams	7
8	Optimum Pull-Up Manoeuvre Load Factor with Height	7
9	Energy Required with varying descent angle for Transition Method B	8
10	Energy Required with varying descent angle for Transition Method C	9

List of Tables

1	Minimum Turn Radius for different load factors	2
2	Energy Consumption per Unit Height for Launch Method 1	5
3	Energy Consumption per Unit Height for Launch Method 2	6
4	Summary of Transition Methods using the final motor and propeller data	9
5	Power Required to Hover from various methods as stated in the Design Process section	10
6	Summary of overall performance	11
7	Summary of Performance Criteria stipulated in the Design Brief	12

1 Introduction

Performance estimation is a critical and integral consideration during the design of an aircraft in order to ensure that the designed product sufficiently meets the design objectives stipulated in the mission brief. This report will investigate three main facets of performance, point performance of the Unmanned Aerial System (UAS) in VTOL configuration, point performance in conventional forward flight and overall mission performance.

Point performance will investigate the instantaneous behaviour of the UAS through parameters such as absolute ceiling, achievable velocities, climb rate and turn rates. These parameters have subsequent integral impacts on other areas of the design such as path planning and object identification. Achieving satisfactory point performance is fundamental to subsequent mission performance and achieving the overall mission profile. Point performance will predominantly be analysed through investigating specific power requirements across the operational envelope.

Mission performance will be evaluated based on the ability of the UAS to perform the mission defined in the initial brief. This includes evaluating if the design meets the range and time constraints as well as ensuring it satisfies the fuel and battery constraints stipulated in initial sizing. An interesting sub-category of the mission performance is minimising the energy consumption of the battery powered stages of flight. Due to the energy density of batteries being much lower than that of hydrocarbon fuels, the performance in this critical stage of flight has a significant impact on the overall weight of the UAS. Therefore this report investigates the optimisation of the transition stage to ensure that the required battery mass does not exceed that of the initial sizing estimates.

Investigating the performance after the final iteration of the design allows for the comparison between initial sizing estimates, modelled results in simulink with a controller applied and flight simulation data. It also provides a benchmark for feedback to different sub-teams in future iterations of the design to enhance the capabilities of the product and hence to make it more marketable.

2 Point Performance

2.1 Climb Performance

The climb performance is an important consideration for our flying liferaft solution. The UAS is expected operate competantly in all weather conditions. Stormy and bad weather phenomena such as windshear or microbursts, which can produce downdrafts of up to 65 m/s in extreme cases [1], facilitate the need for a healthy climb rate. A satisfactory climb rate also shortens the overall mission time as more time can be spent in the comparatively faster cruise.

A specific excess power plot was created using Equation 1 to define the entire envelope of the UAS and can be seen with Figure 1. It is not expected that the UAS will typically operate in the wide range altitudes depicted in the specific excess power plot, however knowing the entire flight envelope of an aircraft is essential for the safe operation of the UAS, for extenuating circumstances and for certification.

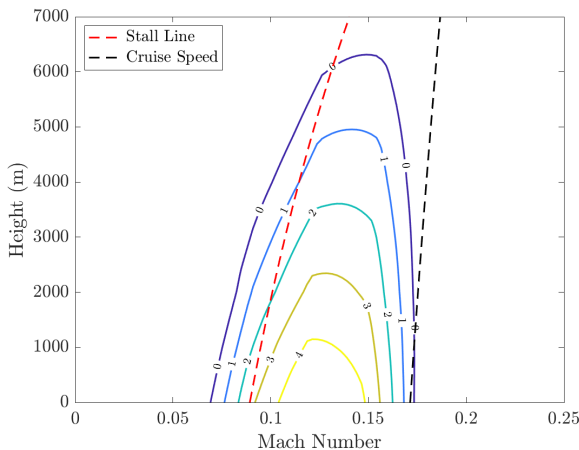


Figure 1: Specific Excess Power Plot

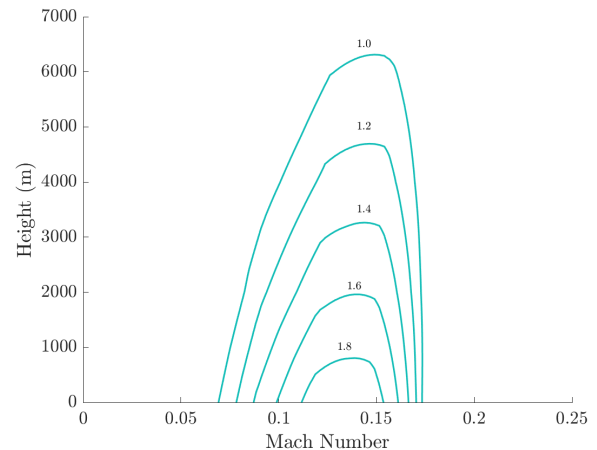


Figure 2: Maximum Turning Load Factor

Figure 1 shows a sea-level climb rate in excess of 4 m/s. The exact maximum climb rate at sea level was found using the specific excess power equation (Equation 1).

$$P_S = V \left[\frac{T}{W} - \frac{qC_{D_0}}{\frac{W}{S}} - \frac{\frac{W}{S}}{q\pi AR e} \right] \quad (1)$$

where q is the dynamic pressure ($\frac{1}{2}\rho V^2$), e is the Oswald efficiency and AR is the Aspect Ratio. When $V = V_{\text{climb}}$, Equation 1 yielded a climb rate of 4.38 m/s or 861 feet per minute (fpm). During initial sizing, a target climb rate of 500 fpm was stipulated. This in turn drove a thrust to weight ratio requirement of 0.22. It can therefore be seen that the UAS comfortably achieves this design specification.

Power and thrust lapse effects alongside the effect of forward velocity on overall thrust were considered through the application of Equation 2 [2].

$$T = SP_{SL} [1.132\sigma - 0.132] \frac{\eta_p}{V_\infty} \quad (2)$$

where SP_{SL} is the shaft power at sea level and η_p is the propeller efficiency factor. The shaft power at sea level was defined by the Propulsion and Systems team to be 8.83 kW. Figure 1 suggests an absolute ceiling of 6327 m, and a maximum Mach number of around 0.17. It also shows how cruise speed is only marginally achievable at low altitudes, with a specific excess thrust of 3.2 N at sea level.

2.2 Turn Performance

The turning performance of the UAS was also analysed. Figure 2 shows specific excess power contours equal to zero at incremental load factors. Figure 2 suggests that the maximum load factor in a sustained turn in the standard operating band (ISA standard conditions) of the UAS is slightly over 1.8 G before being limited by the amount of specific excess power available. In order to maintain a safety margin in the interest of stall prevention and controllability, it is recommended that the UAS does not exceed a turn load factor of 1.8 G at sea level. The amount of specific excess power available is the constraining factor of the turn radius, with the structure being designed for an operational load case of 3 G.

The associated turn radius with the different load factors can be found in Table 1

Table 1: Minimum Turn Radius for different load factors

Turn Load Factor	Radius (m)
1.2	307
1.4	242
1.6	217
1.8	204

The turn radius can be improved by increasing the specific excess power available. This can either be achieved by selecting a more powerful powerplant or by further optimising the propeller blade for increased thrust. However, in the case of this UAS, a more pertinent solution would be to decrease the C_{D_0} of the airframe. Currently the C_{D_0} is comparatively high at 0.066 from StarCCM simulations conducted by the Aerodynamics team. A contributing factor to this is large diameter, short length fuselage that is required to meet payload and deck space constraints. Another significant contributor to the drag is the VTOL superstructure, with exposed pylons, motors and propellers being highly aerodynamically inefficient.

3 Transition Optimisation

The selected parallel hybrid propulsion system requires an independent battery system for the VTOL stages of flight. This battery system must have enough capacity to achieve the necessary manoeuvres for the flight, whilst also achieving the minimum design mass possible in order to maximise payload and ensure the design meets the MTOW constraint in initial sizing. During the design process, it was decided that using an alternator to recharge the batteries using the internal combustion engine was not feasible due to the high weight penalty of the alternator and the variable mission time. Therefore the battery capacity must be sufficient for the entire mission without recharging. Batteries have a poor energy density in comparison to fossil fuels, with the battery selected by Propulsion and Systems team having an energy density of 0.827 MJ/kg compared to

the energy density of fossil fuels at around 46.5 MJ/kg [3] for conventional gasoline. Furthermore, VTOL operations are expensive in terms of energy cost as there is no aerodynamic lifting force contribution. The battery required for VTOL is essential to the function of the UAS, however the battery system is the largest proportion by weight of the drone for a non critical lifesaving system. Therefore the optimisation of the transition from VTOL to Level-flight is of significant importance to minimise the mass of the batteries required, and subsequently maximise payload to improve the functionality and marketability of the product.

In order to analyse and optimise the transition operation, two climbing methods and four transition methods were considered as can be seen in Figure 3

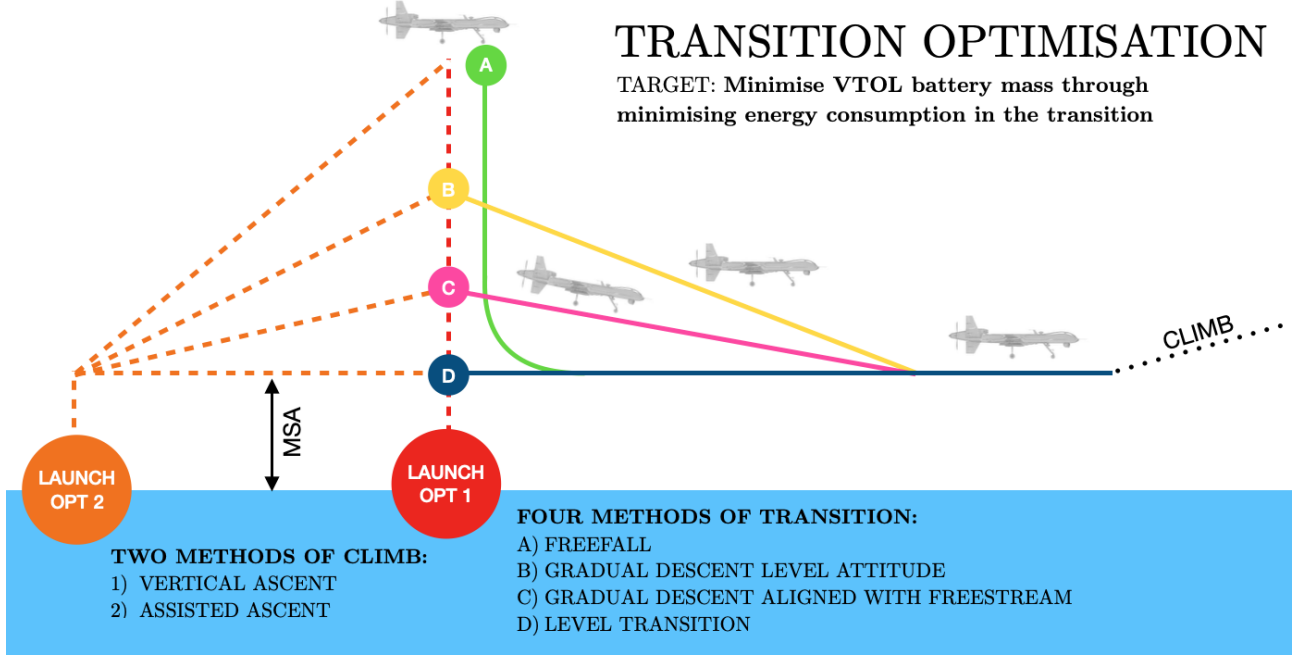


Figure 3: Methods investigated in the Transition Optimisation

Both launch methods (Method 1 and Method 2) require a VTOL climb to a Minimum Safe Altitude (MSA) of 12 m. This was determined as a sensible altitude for the drone to operate safely in all sea-state conditions. An altitude of 12 m should also give sufficient clearance to the nearby vessels and objects, as well as elevating the drone above the steep wind gradient layer that would induce wind shear and therefore aerodynamic and control issues.

The optimisation target of the transition is to minimise the battery mass required to complete the mission profile. This is achieved by selecting the minimum energy launch and transition method. It was assumed that any additional fuel mass for the internal combustion engine could be neglected due to being very small compared to the mass of the batteries. The secondary optimisation target is the time to reach conventional forward flight and the impact on overall mission time. This is to allow for a quick and efficient response to any emergency.

Constraints of the optimisation include:

- Achieving and respecting the transition altitude of 12m.
- Remaining inside the flight envelope (structural and wing operating limits)
- Battery mass should not be more than 10.82 kg as stated in initial sizing
- VTOL motors have a maximum thrust achievable depending on the velocity

3.1 Design Process

For all six cases analysed, the same design process was followed to calculate the energy usage of each method.

1. Equations of motion for each case were created using force diagrams and point mass methods.
2. The acceleration, and consequently velocity and height, were iteratively calculated using kinematic relations with constant acceleration at sufficiently small time intervals of 0.1 seconds.

3. The required vertical thrust component (T_V) was calculated through the equations of motion defined in the Step 1, taking into account any possible vectored force contribution.
4. The required power at every time interval was then extracted from the required vertical thrust. This was done in three different methods as the design process matured:
 - i. Power requirements directly obtained through 1D Axial Momentum Theory [4] were available for some of the performance conditions such as hover.
 - ii. Once the VTOL motor had been selected by the Propulsion and Systems team, but before the actual propeller had been finalised, data from the motor manufacturer was used to create a relationship between the power usage and thrust output using a suitably sized arbitrary propeller. A linear interpolation fit was created to evaluate the power required at any thrust setting.
 - iii. The final power requirement iteration involved using estimated data from the Propulsion and Systems team for the power consumption of the actual propeller with the chosen motor. This was achieved through creating a 2D fit that accepted the required thrust and vertical velocity of the quadcopter to output the required RPM. The RPM was then inputted into a second 2D fit that output the power prediction. A polynomial fit was chosen to match the general trend of the data and to reduce the spurious nature of the power output relationship from BEM theory.
5. The energy was then calculated through integrating the power with respect to time.

3.2 Launch Method

The force diagrams used for each case launch method case can be seen in Figure 4

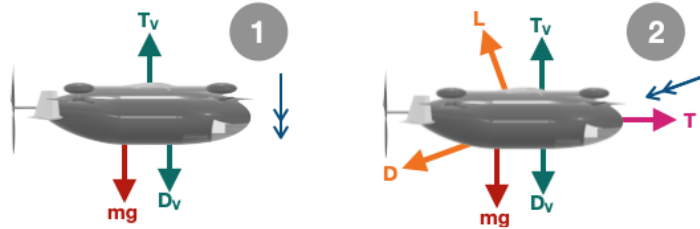


Figure 4: Launch Method Force Diagrams

The energy parameter compared for the launch method was energy per unit height, and was calculated through multiplying the power required by the inverse of the associated climb rate. For all VTOL climb calculations it was considered that the UAS was at sea-level conditions as to be expected in reference to the mission profile.

For Launch Method 2, careful consideration to the maximum achievable angle of incidence possible for the wing. A filter was applied to ensure that only feasible angle of attacks would be possible [-10 degrees to 14.5 degrees].

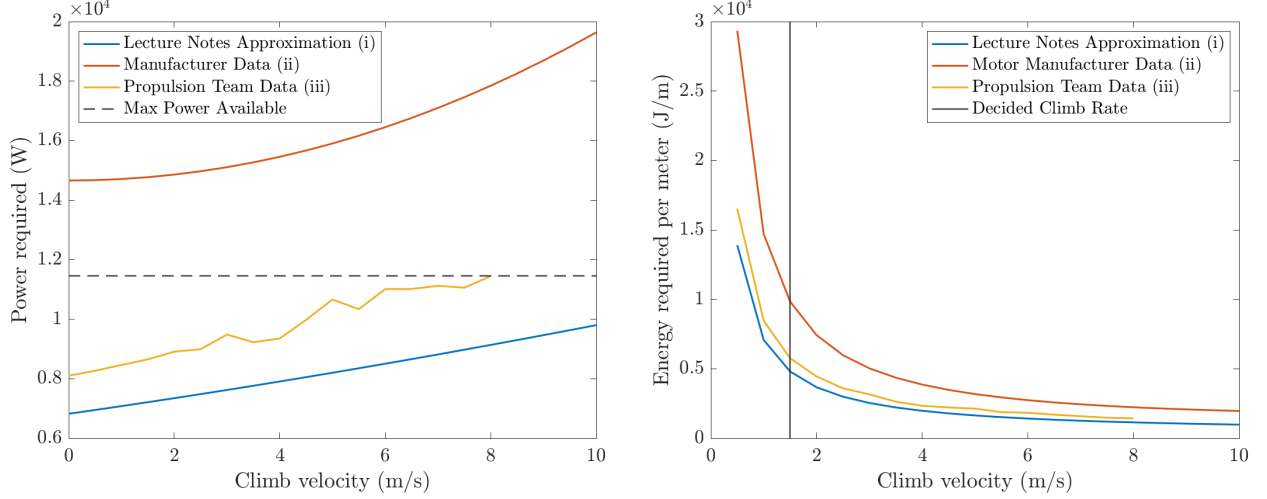
3.2.1 Launch Method 1

Launch Method 1 is defined as a purely vertical ascent, much like a conventional quadcopter using the VTOL motors only. The driving parameter was the vertical speed and was varied between 0 and 4.5 m/s.

The power required to climb at different vertical velocities was initially calculated through Equation 3 [4] (Method 4i, Section 3.1)

$$P = P_h \left[\frac{V_c}{2V_h} + \sqrt{\left(\frac{V_c}{2V_h} \right)^2 + 1} \right] \quad (3)$$

A more sophisticated estimation of the power required for climb was calculated through Method 4ii (Section 3.1). It was assumed that the quadcopter was in a steady climb and that the drag force induced from the vertical velocity could be modelled as a flat plate with the same frontal area as the rotor area (1.13 m^2). The C_{D0} value of this flat plate was taken to be 1.17 [5].



(a) VTOL Power required for different vertical velocities (b) Energy Consumption for different vertical velocities

Figure 5: Climb Method 1

It was decided that a vertical climb velocity of 1.5 m/s would be selected to calculate the energy required per unit height in the VTOL mode. This was decided based on a reasonable estimate of 8 seconds to reach the minimum safe altitude of 12m in consultation with the simlink control simulations. Although Figure 5a suggests that a climb rate up to 8 m/s is possible, which in correspondence with Figure 5b would give the minimum energy consumption, it is not reasonable to assume the quadcopter can accelerate sufficiently in the distance available. A vertical speed of 1.5 m/s mostly overcomes the initial asymptotic behaviour of energy usage as can be seen in Figure 5 and was therefore determined an appropriate compromise.

Table 3 shows a comparison between the energy requirements per unit height using methods specified in the design process at a climb rate of 1.5m/s.

Table 2: Energy Consumption per Unit Height for Launch Method 1

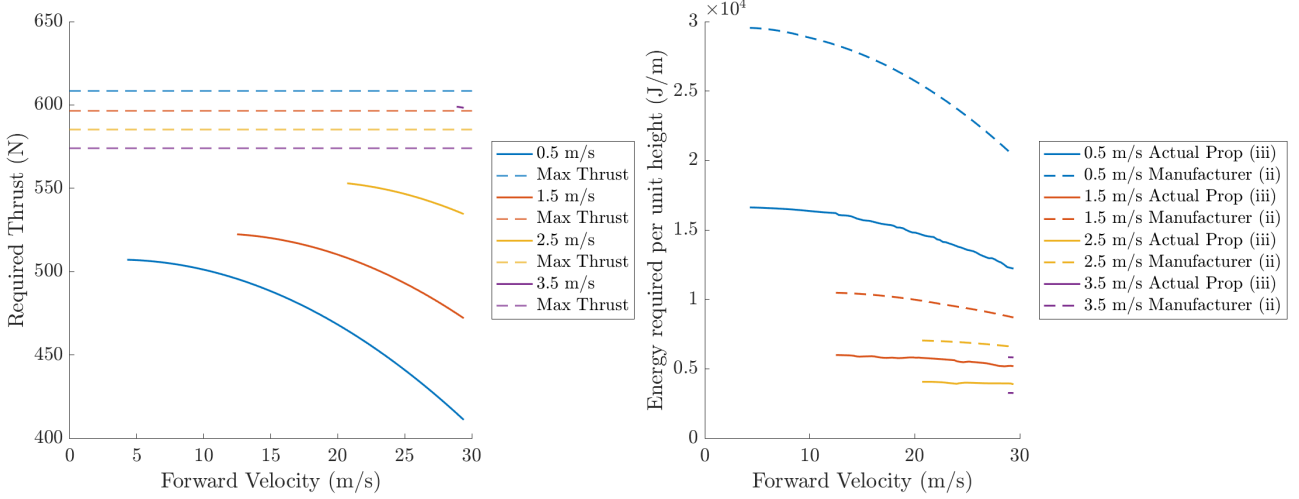
Method	Energy Per Unit Height (kJ/m)
Lecture Notes Approximation (i)	4.81
Motor Manufacturer Data (ii)	9.85
Final Motor and Propeller Data (iii)	5.77

Table 3 shows a noticeable discrepancy between manufacturer data and data yielded from the Propulsion and Systems team. It can be argued that energy usage is expected to be less due to the propeller being optimised for the specific UAS. However the more likely reasoning behind the discrepancies are real world losses which are included in the experimental data but not encapsulated in the the optimised propeller such as aerodynamic and motor inefficiencies. Table 3 also shows how the Lecture Note approximations are the lowest of the energy methods. This is likely due to the simplifications made in the 1D axial momentum theory.

3.2.2 Launch Method 2

In order to evaluate the energy cost for Launch Method 2, two parameters were varied; forward velocity between 0 m/s and stall speed (V_S) and vertical velocity between 0 and 4.5 m/s.

Figure 6b shows how increasing forward velocity decreases the energy required per unit height. Conversely, Figure 6a shows how increasing the vertical velocity actually increases the vertical thrust required by the VTOL propellers. This is to be expected as increasing the vertical velocity component effectively decreases the angle of attack seen by the wing therefore it creates less lift. However when considering total energy cost, this effect is upscaled by the reduced time to climb to altitude. The operating conditions and envelope of the wing were respected by filtering out infeasible angles of attack as defined by the Aerodynamics team. A maximum angle of attack of 14.5 degrees and minimum angle of -10 degrees was used, with an assumption of a linear lift curve



(a) VTOL Thrust required for different forward and vertical velocities (b) Energy Consumption for different forward and vertical velocities

Figure 6: Climb Method 2

slope (4.1 /rad), a zero lift angle of attack of -2.71 degrees and a wing setting angle of 1.36 degrees. As can be seen in Figure 4, it was assumed that the quadcopter stayed level with the global horizon during the climb. Inducing a positive angle of attack would increase the lift produced by the wing, however would also create a large horizontal drag component from the thrust vectoring. Another important consideration is the maximum thrust that the motors can produce, which can be seen in Figure 6a. This constraint eliminates climb rates in excess of 3.5 m/s to see any aerodynamic benefit from the lifting surfaces.

A vertical speed of 1.5 m/s was chosen to allow for direct comparison with Climb Method 1 stated in Section 3.2.1. Furthermore, consulting Figure 6 it provides a good trade off between energy reduction and range of possible forward velocities. This is an important consideration as the UAS must accelerate to the minimum forward velocity and will use energy in doing so. For a vertical speed of 1.5 m/s the energy consumption per unit height can be seen in Table 3

Table 3: Energy Consumption per Unit Height for Launch Method 2

Method	Energy Per Unit Height (kJ/m)
Motor Manufacturer Data (ii)	8.70
Final Motor and Propeller Data (iii)	5.18

3.2.3 Comparison

Launch Method 2 is the preferred climb method. This is predominantly due to the reduced energy consumption per unit height gained. However another important consideration is the impact of controllability on the UAS. Forward flight of the UAS allows for airflow to flow over the conventional control and stability surfaces. This makes for a more stable and controllable platform. Launch Method 2 also involves a horizontal displacement, which in turn shortens the required cruise distance to target and is more time efficient to the overall mission profile.

Therefore for the climb up to the MSA, Climb Method 1 must be used at an energy consumption of 5.77 kJ/m, however for any additional climb required to complete the transition then Climb Method 2 is to be used at an energy consumption of 5.18 kJ/m at a climb rate of 1.5 m/s.

3.3 Transition Methods

The force diagrams used for the transition equation of motions can be seen in Figure 7

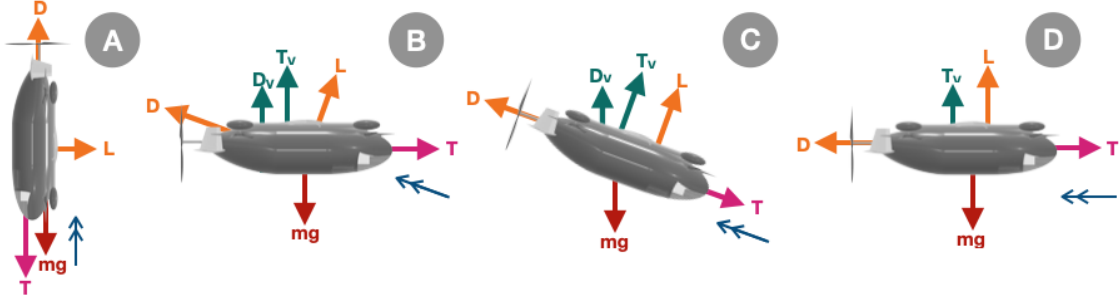


Figure 7: Transition Method Force Diagrams

3.3.1 Transition Method A

Transition Method A involves a vertical 'nose-dive' descent, followed by a 90 degree pull up manoeuvre. The vertical dive equations of motion were established, with both Thrust and Drag being considered as functions of velocity. The pull up manoeuvre load factor was varied in order to find the optimum load factor for minimum height loss. The pull up manoeuvre speed was subsequently defined as the 1.1 times the stall speed for the specified load factor. The 1.1 times safety factor was used to prevent an accelerated stall condition. It was assumed that during the descent and the pull up the wing was able to operate at $C_{L_{max}}$. The radius of the pull-up manoeuvre was calculated through Equation 4.

$$R_{\text{Pull Up}} = \frac{V^2}{g(n-1)} \quad (4)$$

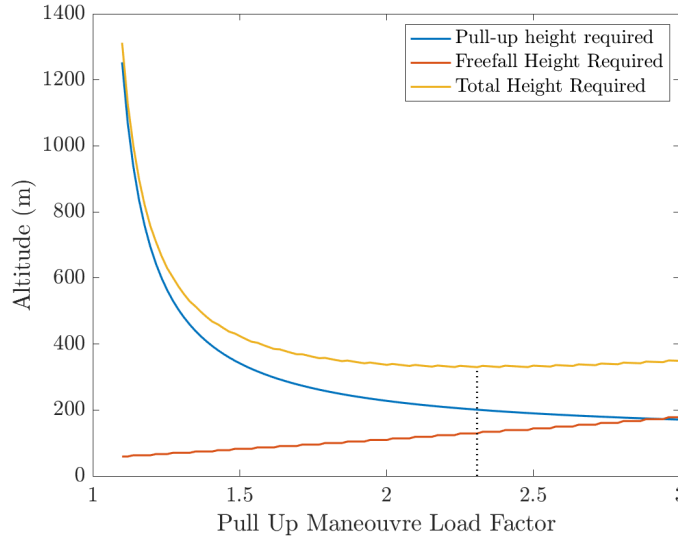


Figure 8: Optimum Pull-Up Manoeuvre Load Factor with Height

Figure 8 shows how a minimum height loss is achieved with a pull up load factor of 2.31 G, with an altitude loss of 330 m.

As determined in Section 3.2.3, Climb Method 2 is best suited for climbing to the required altitude of 330 m and would require 1708 kJ of energy. This transition method has a higher risk factor than other transition methods and is more structurally demanding due to the high dive speed and the load factor pulled during the pull-up.

3.3.2 Transition Method B

Transition Method B uses a gradual descent, with the quadcopter remaining level to the global horizon. This creates a freestream velocity at an angle of incidence the same as the descent angle. This method is beneficial as it allows the wing to operate at a higher angle of attack and therefore reducing the velocity that needs to be achieved before the drone can support it's own weight and continue under conventional flight. However it requires a higher starting height to respect the

minimum safe altitude. The descent angle was varied from 1 degree to the stall angle of the wing (14.5 degrees).

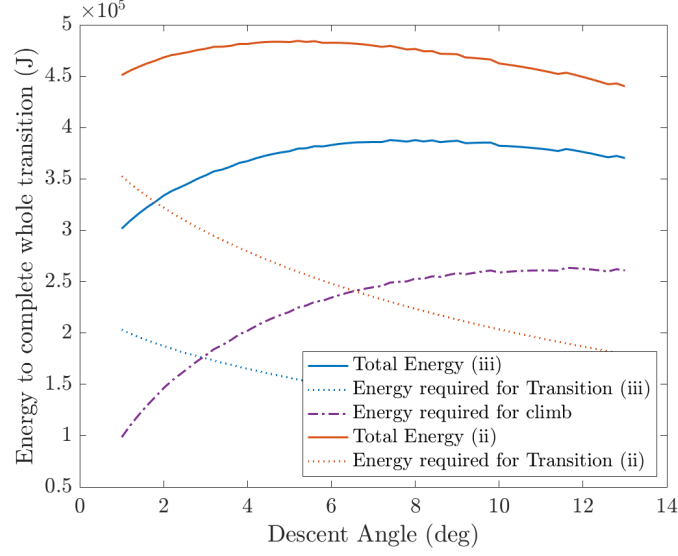


Figure 9: Energy Required with varying descent angle for Transition Method B

Figure 9 suggests that a minimum energy consumption for transition is achieved through as low a descent angle as possible. This is mostly due to the climb required to gain the necessary height for the descent angle and the expensiveness of the climb operation as can be seen with the semi-dotted purple line and its influence on the overall energy consumption. With a descent angle of 1 degree, the energy required to complete the transition was estimated to be 302 kJ.

A possible reason for the discrepancy between the power and energy consumption between manufacturer data (Method ii) and our final propeller design (Method iii) is that at low RPM values the blade is stalled and therefore the data set is not well defined. Consequently erroneous and missing values have been removed. This has an impact on the subsequent integration to yield the energy and therefore giving an under-reading of the true result. However, this is not likely to have a drastic effect on the results due to the small contribution of energy in the final stages of the transition.

3.3.3 Transition Method C

Transition Method C prescribes a gentle descent with a pitch angle equal to that of descent. This allows for the thrust of the quadcopter to be utilised as forward propulsion. It was assumed that the UAS would operate aligned with the freestream velocity with no angle of incidence. The descent angle was varied from 1 degree to 21 degrees as a sensible pitch angle for a quadcopter after consulting control.

Likewise to Transition Method B, the descent angle requires an increased start height which drastically negatively impacts the overall energy consumption as can be seen in Figure 10.

Also similar to Transition Method B is the trend that a smaller descent angle is more beneficial for the concern of energy consumption. With a descent angle of 1 degree, the energy required to complete the transition was estimated to be 380 kJ.

3.3.4 Transition Method D

Transition Method D involves a level acceleration of the drone until it can sustain conventional forward flight. In this transition method, the quadcopter motors purely provide vertical lift and are not inclined to produce any usable forward thrust. This is due to any forward thrust then requiring a negative pitch angle which in turn reduces the lift generation of the main wing.

This transition method was predicted to require 224 kJ to complete the transition. Table 4 shows how this compares with the previous transition methods. It is the lowest of all the methods due to the lack of climb required to complete the transition.

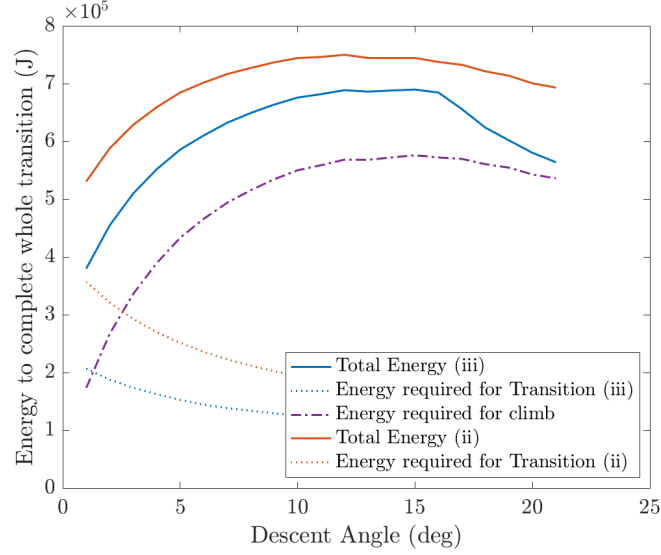


Figure 10: Energy Required with varying descent angle for Transition Method C

Table 4: Summary of Transition Methods using the final motor and propeller data

Transition Method	Energy Required to Complete Transition (kJ)
A	1710
B	302
C	380
D	224

3.4 Further Effects

3.4.1 Ground Effect

The aerodynamic effect of ground effect would greatly decrease the energy required to sustain the weight of the drone. However due to the MSA this was not considered as a viable option of transition for operational reasons.

3.4.2 Effective Translational Lift and Transverse Flow

Effective Translational Lift is an interesting aerodynamic phenomena that was investigated and subsequently integrated into the mission profile. As a helicopter moves forward, rotor efficiency increases with forward velocity until a certain threshold. Literature suggests this minimum energy consumption occurs when the forward velocity is as such that the rotor's are be unaffected by their own wake [6]. For conventional helicopters, this speed is around 16-24 knots [?].

Transverse flow is the effect of inclining the rotor area resulting in an increased mass flow rate through the rotor area due to the forward velocity component, increasing effective thrust [6].

It was therefore decided that the payload delivery would be conducted at 10 m/s in order to take advantage of these aerodynamic effects with the aim of reducing energy consumption during this mission stage.

3.4.3 Stall Speed

A preliminary investigation into the effect of reducing stall speed on battery mass was conducted. A reduced stall speed means the VTOL motors are not required to provide thrust for as long and therefore reduces the energy consumption for transition. It was found that a reduction in stall speed of 2 m/s reduced the battery weight by a total of 40 grams. Seeing that the battery is already limited by the voltage and current requirements it was decided that decreasing the stall speed at the expense of aerodynamic performance was not beneficial.

4 VTOL Performance

4.1 Minimum Hover Power Requirement

The minimum power requirement to hover was initially approximated early on in the design process using Equation 5 (Method i) [4]

$$P_{\text{hover, min}} = \frac{W^{\frac{3}{2}}}{\sqrt{2\rho A}} \quad (5)$$

where A is the rotor area. This method uses 1D axial momentum theory

The chronological design process stated in Section 3.1 was then followed, yielding power requirement results shown in Table 5

Table 5: Power Required to Hover from various methods as stated in the Design Process section

Method	Energy Required to Hover (kJ/s)
Lecture Notes Approximation (i)	6.82
Motor Manufacturer Data (ii)	14.7
Final Motor and Propeller Data (iii)	8.10

4.2 VTOL Maximum Ceiling

The absolute ceiling was calculated through Equation 6

$$\sigma_{\text{max}} = \left(\frac{P_{h0}}{P_{\text{max}}} \right)^2 \quad (6)$$

where P_{h0} is the power required to hover at sea level, and P_{max} is the maximum power available. Equation 6 yields a maximum density ratio of 0.49 which corresponds to an altitude of 6800 m. This however is unlikely to be achievable. A significant reason is that Equation 6 does not consider rotor profile power or transmission loss. It also only evaluates the hover and power requirements at sea level, therefore the available power is likely to drastically decrease with increasing altitude. This result is therefore not to be used reliably. The hover ceiling however is not a significant concern for this particular application as all VTOL operations are close to the water.

5 Mission Performance

All elements of the mission profile were considered in light of improved accuracy of elements from design iterations. The mission performance was separated into the component parts of the mission profile.

5.1 Assumptions in Mission Performance

5.1.1 VTOL Takeoff and Transition

As discussed in Section 3.2 it was assumed that the UAS in VTOL mode travels at an average vertical velocity of 1.5 m/s throughout the VTOL climb. Results from Section 3.3 yield that Transition Method D is the most efficient transition method. During the transition periods, the internal combustion engine was modelled to consume the same fuel as at cruise due to the lack of propulsion data at low speeds. It was assumed that the VTOL motor and propeller did not have a minimum operating rpm and subsequent minimum thrust output. Results from Section 3.2 and Section 3.3 have been populated in Table 6.

The alternate transition was modelled to slow down to 10 m/s. This not only is the drop velocity, but was also deemed to be a sufficiently slow speed for quadcopter behaviour and dynamics to slow the UAS to zero velocity without additional aerodynamic effects. It was modelled as a level deceleration, with the VTOL propellers subsidising the reducing lift as the UAS decelerates. The drag of the windmilling propeller was modelled through assuming a disc of area of that of the pusher propeller.

5.1.2 Climb, Cruise and Descent to Loiter

The height of cruise was iteratively optimised. Increasing the cruise height increases climb and descent times, but has the potential to decrease overall mission time through the benefit of the trade-off between True Air Speed and Indicated Air Speed. Furthermore varying height also has

an effect on fuel consumption of the mission. The best rate of climb speed V_y was calculated to be 37.2 m/s using Equation 7

$$V_{\text{climb}} = V_{\text{imD}} \left[\sqrt{\frac{\tau}{3} + \frac{\sqrt{\tau^2 + 3}}{3}} \right] \quad \text{where; } \tau = \frac{T}{D_{\text{min}}} \quad (7)$$

It was decided that for the first cruise that the main optimisation target would be time to minimise time to the target and consequentially maximise survivability.

It was found that increasing cruise altitude increased mission time. This is due to the slower climb velocity and the relatively short range which means that the True Airspeed Benefit is limited. It was decided that the UAS should cruise at the loiter height. This is to allow for a simpler mission profile, reducing the number of flight phases that have to be controlled, whilst also to allowing for the UAS sensor to patrol during the cruise period as well.

For the second cruise, time was no longer the most important parameter. Therefore the second cruise was optimised for minimum fuel consumption. This resulted in a second cruise speed of 47.8 m/s corresponding to the best lift to drag ratio speed of the aircraft. Due to also no longer having to detect people in the water, the cruise altitude was set as the minimum safe altitude resulting in no climb or descent component.

The specific fuel consumption of the engine chosen by the Propulsion and Systems team was averaged to be 590 g/kWh. Using the power requirements for each major stage of flight, the specific fuel consumption was found; with the first cruise being 4.248 kg/hr, second cruise 2.773 kg/hr and loiter of 1.947 kg/hr. There was no dedicated specific fuel consumption for the transition and climb phase, therefore the highest specific fuel consumption (Cruise 1) was used.

5.1.3 Loiter and Descent to drop altitude

The loiter and descent to drop altitude determined from the mission brief of 30 minutes from launch to providing relief. As can be seen in Table 6, the time to loiter is 21.3 minutes. This therefore left 8.7 minutes for target detection and descent to achieve the mission brief.

5.1.4 Dropping the Payload and VTOL Landing

During the drop phase it was assumed the energy consumption was the same as at hover despite the aerodynamic effects discussed in Section 3.4.2 considering a worst case scenario.

Landing energy requirement was calculated through assuming an 8 second recovery time (same as that of take off) and assumed a worst case scenario of the hover energy requirement to accommodate for control inputs on descent.

5.2 Summary of Mission Performance

Table 6: Summary of overall performance

Mission Segment	Time (s)	Battery Required (kg)	Fuel Required (kg)
VTOL Takeoff to MSA	8.00	0.0837	0
Transition to Conventional Forward Flight	59.8	0.270	0.0696
Climb	79.4	0	0.0937
Outbound Cruise	1130	0	1.33
Descent to Loiter Altitude	0	0	0
Loiter and Descent to Drop Altitude	523	0	0.282
Transition to VTOL mode for Payload Delivery	31.9	0.246	0
Hover for Payload Delivery	30	0.294	0
Transition to Conventional Forward Flight	59.8	0.270	0.0696
Climb	0	0	0
Return Cruise	1464	0	1.09
Descent to MSA	0	0	0
Transition to VTOL mode for Landing	31.9	0.246	0
VTOL Landing	8.00	0.0784	0
TOTAL	3430	1.49	2.94

Aside from the energy capacity of the battery, the minimum battery size was also constrained by the voltage and current requirements of the system as detailed with the Propulsion and Systems team. The minimum battery mass calculated by the Propulsion and Systems team to meet the voltage and current requirements was determined to be 5.5 kg. This leaves an excess capacity of 4.01 kg which allows for an additional 410 seconds of hover. This improves mission flexibility and gives a reassuring safety margin for the drone's operation and increases mission flexibility.

6 Conclusion

The UAS was found to meet the majority of performance metrics stipulated in the mission brief, however did not manage to achieve all the criteria.

Table 7: Summary of Performance Criteria stipulated in the Design Brief

Design Criteria	Target	Actual	Met?
Fuel Mass	1.96 kg	2.94 kg	NO
Battery Mass	10.8 kg	1.49 kg*	YES
Climb Performance	500 fpm	861 fpm	YES
Cruise Speed	58.3 m/s	58.3 m/s	YES
Time to Loiter	20 mins	23.1 mins	NO

* *Minimum mass required for energy not taking into account voltage and current requirements*

The point performance was deemed to be adequate for the mission in question, however the turn rate could be improved. A method of improving the turn rate is to increase the specific excess power available by increasing the available thrust of the engine. However the more prevalent method of increasing specific excess power would be to reduce the C_{D0} of the UAS which is comparatively very high.

The VTOL performance was also deemed to be satisfactory for the mission profile. Analysis conducted suggested a maximum possible climb rate of 8 m/s which is more than adequate for the mission requirements. The absolute ceiling was determined to be 6800 m, however it was decided this was not a reliable result due to neglecting inefficiencies and propeller performance at altitude.

Out of the two climb methods investigated before forward flight is achieved, a gentle climb method (Method 2) was chosen to be both the most time and energy efficient at 5.18 kJ/m compared to 5.77 kJ/m for VTOL climb (Method 1). However Method 1 must be used to reach the Minimum Safe Altitude.

Out of the four transition methods investigated, Transition Method D with a level transition was deemed most efficient using 224 kJ to complete the transition. It was assumed that the VTOL motor and propeller did not have a minimum operating rpm and subsequent minimum thrust output. This was validated by Transition Method B and C suggesting that a smaller descent angle is more efficient.

The design was found to not achieve the required range with the original fuel estimation of 1.96 kg. This however was not deemed to be a significant issue as the fuel tank has the capacity for 5.5 kg of fuel and the initial battery mass was over-estimated. The minimum required battery mass for energy consumption was found to be less than the minimum mass for voltage and current requirements giving excess capability for the UAS. Initial sizing suggested a combined fuel and battery mass of 12.77 kg whereas performance estimates suggest a fuel and battery mass of 8.44 kg.

Overall, the design was found to meet some, but not all of the criteria defined in the initial mission profile. Suggestions for future design iterations include increasing the specific excess power or, more pertinently, reducing the C_{D0} value to improve all aspects of performance.

Improvements to this report would include using an optimiser software, such as MATLAB's `fmincon` function, to validate the results of the optimisation study. The specific fuel consumption values used could also be improved as they were a close approximate of the actual engine data, therefore higher accuracy and fidelity in the fuel consumption would lead to more accurate predictions. Finally the C_{D0} estimate for the drag contribution by propeller areas should be refined from the crude approximation of a flat plate. This could be achieved through utilising experimental data.

References

- [1] N. W. Service. What is a microburst? [Online]. Available: <https://www.weather.gov/ama/microbursts>
- [2] V. Tech. Performance. [Online]. Available: <http://www.dept.aoe.vt.edu/~lutze/AOE3104/thrustmodels.pdf>
- [3] E. Toolbox. [Online]. Available: https://www.engineeringtoolbox.com/fossil-fuels-energy-content-d_1298.html
- [4] E. Levis, “Introduction to aerospace,” 2019.
- [5] E. Toolbox. Drag coefficient. [Online]. Available: https://www.engineeringtoolbox.com/drag-coefficient-d_627.html
- [6] U. S. Army. Fundamentals of flight. [Online]. Available: <https://irp.fas.org/doddir/army/fm3-04-203.pdf>

Appendix

Performance Script

Kristoffer Ahlner FLR22 GDP

Defining Parameters

```
%HOUSEKEEPING
clear
clc
close all

set(groot, 'defaulttextinterpreter', 'latex');
set(groot, 'defaultAxesTickLabelInterpreter', 'latex');
set(groot, 'defaultLegendInterpreter', 'latex');

%IMPORTING THE VARIABLES
Aero = table2struct(readtable('variables.xlsx', 'Range', 'A2:Z3', 'ReadVariableNames', true), true);
Power = table2struct(readtable('variables.xlsx', 'Range', 'A17:Z18', 'ReadVariableNames', true), true);
Struct = table2struct(readtable('variables.xlsx', 'Range', 'A12:Z13', 'ReadVariableNames', true), true);
Vel = table2struct(readtable('variables.xlsx', 'Range', 'A7:Z8', 'ReadVariableNames', true), true);
Mission = table2struct(readtable('variables.xlsx', 'Range', 'A43:Z44', 'ReadVariableNames', true), true);
VTOLMotor = readtable('variables.xlsx', 'Range', 'A21:E29');
VTOLMotor.Thrust = VTOLMotor.Thrust/1000;
ICEMotor = readtable('variables.xlsx', 'Range', 'A32:B40');

load('max_thrust_V_h.mat')
load('max_thrust_v_climb.mat')
load('VTOLData (2).mat')

%DEFINING CONSTANTS
g = 9.81;
rho = 1.225;
tol = 0.0001;
width = 1.5;
fontsize = 14;

%PERFORMANCE CONSTANTS
A = 0.5*rho*Aero.Sref*Aero.C_D0;
B = ((Aero.W_des*g)^2)*(0.5*rho*Aero.Sref*pi*Aero.AR*Aero.e);
n = 1;

%VELOCITIES
Vel.V_md = (((B*(n^2))/A)^0.25)/1.944; % Converting to m/s from knots
Vel.V_cl = 1.3*Vel.V_s; % Approximate Climb velocity

%SPECIFIC FUEL
C.power = Power.sfc;
C.cruise = C.power*Vel.V_cruise*3.28084/(550*Power.Eta_Pcruise)/3600;
C.cruise2 = C.power*Vel.V_cruise2*3.28084/(550*Power.Eta_Pcruise)/3600;
C.loiter = C.power*Vel.V_loiter*3.28084/(550*Power.Eta_Ploiter)/3600;
% C.max = 0.245; %kg/kWh
C.max = C.cruise;
```

Initial Mission Performance

```
%% INITIAL BASELINE CONFIGURATION ESTIMATES
% First cruise
Wi_Wf.c1 = 0.9846; %Weight required for first cruise
W0_Wf.c1 = Wi_Wf.c1; %Total weight fraction

range.c1 = (Vel.V_cruise*Aero.LoverD_cruise_init/C.cruise)*log(Wi_Wf.c1^-1)/1000;

% Loiter
Wi_Wf.l = 0.9923; %Weight fraction required for loiter
W0_Wf.l = Wi_Wf.c1*Wi_Wf.l;

loiter_endurance = (Aero.LoverD_loiter_init/C.loiter)*log(Wi_Wf.l^-1)/60 %Minutes

loiter_endurance = 10.5473

%Second cruise
Wi_Wf.c2 = 0.9846; %Weight fraction required for second cruise
W0_Wf.c2 = Wi_Wf.c1*Wi_Wf.l*Wi_Wf.c2;

range.c2 = (Vel.V_cruise*Aero.LoverD_cruise_init/C.cruise)*log(Wi_Wf.c2^-1)/1000; %Show

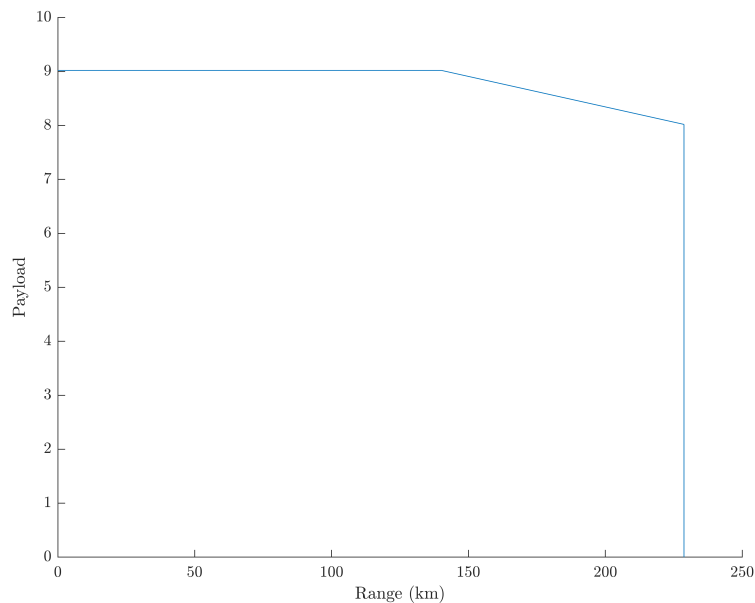
range.total = range.c1 + range.c2

range = struct with fields:
    c1: 70.0910
    c2: 70.0910
    total: 140.1820

%Range with constant Cl - varying height
Vel.vbar = Vel.V_cruise/Vel.V_md;

%% RANGE IF DO NOT TAKE ALL OF THE PAYLOAD
range.unitfuel = (Vel.V_cruise*Aero.LoverD_cruise_init/C.cruise)*log(Aero.W_des/(Aero.W_des - Struct.payload));
range.max = range.total + range.unitfuel*Struct.excessfuelmass;

figure
xlabel('Range (km)')
line([0,range.total],[Struct.Payload,Struct.Payload])
hold on
line([range.total,range.max],[Struct.Payload,Struct.Payload-Struct.excessfuelmass])
line([range.max,range.max],[0,Struct.Payload-Struct.excessfuelmass])
hold off
ylabel('Payload')
```



Point Performance

Climb Performance

```
% Max Rate of Climb Airspeed (Vy)
```

```
Aero.D_min = 2*sqrt(A*B);
```

```
towel = max_thrust_V_h(Vel.V_cl,15)/Aero.D_min;
```

```
Vel.vbar_cl = sqrt((towel/3)+(sqrt(towel^2 + 3)/3));
```

```
Vel.V_cl = Vel.vbar_cl * Vel.V_md;
```

```
T = max_thrust_V_h(Vel.V_cl,15);
```

```
C_D = Aero.C_D0 + Aero.C_Lclimb^2/(pi*Aero.AR*Aero.e)
```

```
C_D = 0.1336
```

```
D = 0.5*1.225*Aero.Sref*C_D*(Vel.V_cl).^2;
```

```
WS = Aero.W_des*9.81/Aero.Sref;
```

```
K = 1/(pi*Aero.e*Aero.AR);
```

```
climbrate.Fixed_wing = Vel.V_cl * ((T/(Aero.W_des*9.81))-(0.5*1.225*Vel.V_cl^2*Aero.C_L
```

```
climbrate.Fixed_wing_fpm = climbrate.Fixed_wing*3.28*60
```

```
climbrate = struct with fields:
```

```
Fixed_wing: 4.3774
```

```
Fixed_wing_fpm: 861.4625
```

```
gamma = asind(climbrate.Fixed_wing/Vel.V_cl);
```

```

array.height = linspace(0,7000,500); %m
array.mach = linspace(0,0.25,100);

[~, array.a , ~ , array.density] = atmosisa(array.height);
array.sigma = array.density/1.225;

array.cruise = Vel.V_cruise./array.a;

SEP.VELOCITY = array.a'*array.mach; %Vert = height, Hor = mach
array.stall = Stall_func(array.height,Aero.C_Lmax,Aero.W_des,Aero.Sref);

[SEP.MACH, SEP.HEIGHT] = meshgrid(array.mach , array.height);
[~, SEP.DENSITY] = meshgrid(array.mach , array.density);
[~,SEP.SIGMA] = meshgrid(array.mach,array.sigma);
SEP.THRUST = max_thrust_V_h(SEP.VELOCITY,SEP.HEIGHT);
SEP.DRAG = Drag_func(SEP.VELOCITY,SEP.SIGMA,Aero.C_D0,Aero.W_des,Aero.AR,Aero.e,Aero.S

SEP.levels = [0:4]; %m/s

K = 1/(pi*Aero.AR*Aero.e);
q = 0.5.*SEP.DENSITY.*SEP.VELOCITY.^2; %dynamic pressure
n = 1;

SEP.CD0 = ones(size(SEP.VELOCITY)).*Aero.C_D0;

SEP.Ps = SEP.VELOCITY.*((SEP.THRUST/(Aero.W_des*g))-((SEP.DRAG)./(Aero.W_des*9.81)));

[testheight,testmach] = max_climb_func(SEP.Ps,SEP.levels,array.height,array.mach);
testheight(2:6) = testheight;
testheight(1) = 0;
testmach(2:6) = testmach;
testmach(1) = Vel.V_cl/340;

figure
[SEP.C,SEP.h] = contour(SEP.MACH,SEP.HEIGHT,SEP.Ps,SEP.levels,'LineWidth',width); %,'SI
xlabel('Mach Number','Interpreter','latex')
ylabel('Height (m)','Interpreter','latex')
clabel(SEP.C,SEP.h,SEP.levels,'Interpreter','latex')
set(gca,'TickLabelInterpreter','latex')
hold on
a1 = plot(array.stall,array.height,'--r','LineWidth',width); M1 = "Stall Line"

M1 =
"Stall Line"

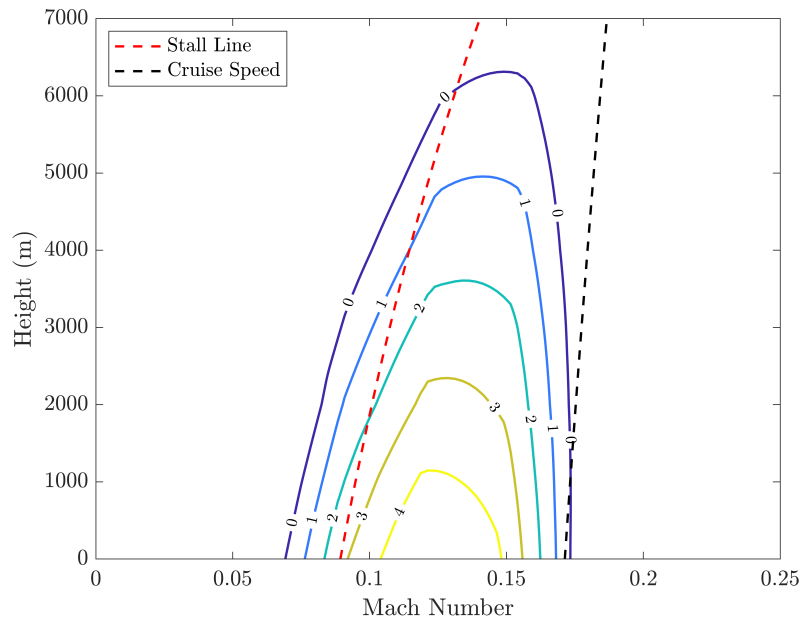
a2 = plot(array.cruise,array.height,'--k','LineWidth',width); M2 = "Cruise Speed"

M2 =
"Cruise Speed"

legend([a1,a2], [M1, M2],'Location','northwest');
% plot(testmach,testheight,'--k')
hold off
set(gca,'fontsize', fontsize)

```

```
saveas(gcf,'SEP.png')
```



```
%Verification
```

```
sigma_maxalt = (((2*Aero.W_des*9.81)/max_thrust_V_h(Vel.V_md,0))*(Aero.C_D0/(pi*Aero.e:
maxalt.conventional = testheight(1)
```

```
maxalt = struct with fields:
    conventional: 0
```

```
%Checking the maximum thrust required
```

```
CL_quad0 = C_L_func(0,Aero.a_w,Aero.i_w,Aero.ZLA)
```

```
CL_quad0 = 0.3305
```

```
C_D = Aero.C_D0 + CL_quad0^2/(pi*Aero.e*Aero.AR)
```

```
C_D = 0.0733
```

```
Vs_quad0 = sqrt((2*Aero.W_des*g)/(Aero.Sref*1.225*CL_quad0))
```

```
Vs_quad0 = 59.3460
```

```
D = 0.5*1.225*Aero.Sref*Vs_quad0^2*C_D
```

```
D = 112.0167
```

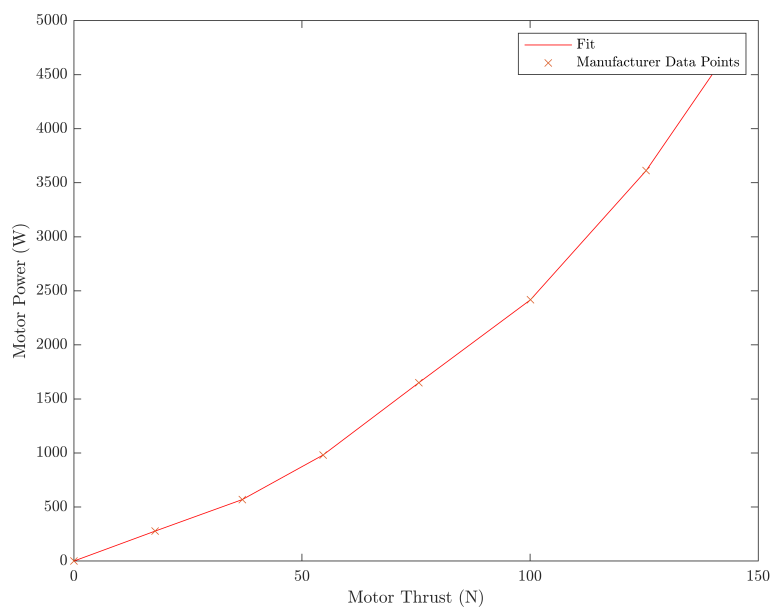
```
max_thrust_V_h(Vs_quad0,0)
```

```
ans = 110.2260
```

Motor Performance

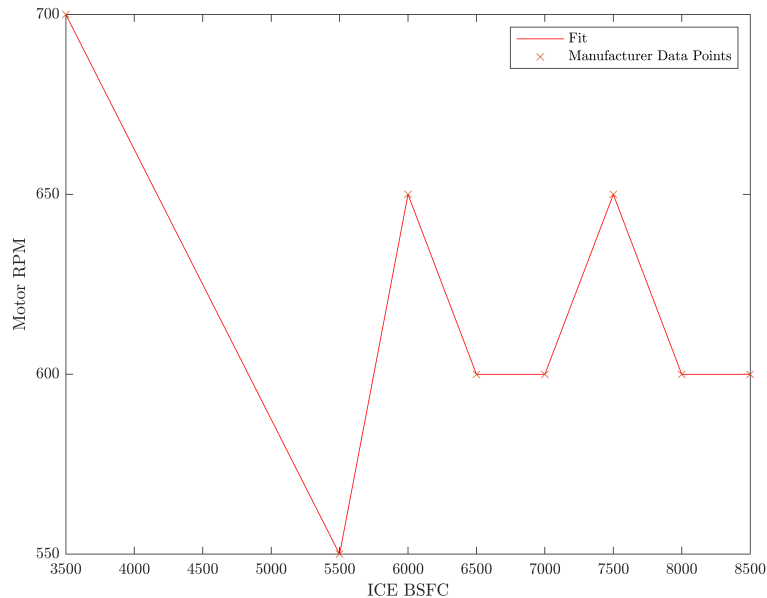
```
% VTOL Motors
motor_fit = fit(VTOLMotor.Thrust*9.81,VTOLMotor.Power,'linearinterp');

figure
plot(motor_fit)
hold on
plot(VTOLMotor.Thrust*9.81, VTOLMotor.Power,'x')
hold off
xlabel('Motor Thrust (N)')
ylabel('Motor Power (W)')
legend('Fit','Manufacturer Data Points')
```



```
% Pusher motor fuel consumption
ICEMotor_fit = fit(ICEMotor.RPM,ICEMotor.BSFC,'linearinterp');

figure
plot(ICEMotor_fit)
hold on
plot(ICEMotor.RPM,ICEMotor.BSFC,'x')
hold off
xlabel('ICE BSFC')
ylabel('Motor RPM')
legend('Fit','Manufacturer Data Points')
```



```
[MESH.RPM, MESH.V] = meshgrid(rpm_range,V_inf_range);

thrust_matrix(:,1:9) = [];
power_matrix(:,1:9) = [];
MESH.V(:,1:9) = [];
MESH.RPM(:,1:9) = [];
thrust_matrix = thrust_matrix';
thrust_matrix = thrust_matrix(:);
power_matrix = power_matrix';
power_matrix = power_matrix(:);
MESH.V = MESH.V';
MESH.V = MESH.V(:);
MESH.RPM = MESH.RPM';
MESH.RPM = MESH.RPM(:);

VTOL_motor_fit1 = fit([thrust_matrix,MESH.V],MESH.RPM,'linear'); %(Thrust,velocity)
VTOL_motor_fit2 = fit([thrust_matrix,MESH.RPM],power_matrix,'linear'); %(Thrust,rpm)
```

Warning: Duplicate x-y data points detected: using average of the z values.

Helicopter performance

```
%Minimum power to hover
Power_req.hover_unittime_mi = ((Aero.W_des*9.81)^(3/2))/(sqrt(2*Power.VTOL_A*1.225));
Vel.Vh = sqrt((Aero.W_des*9.81)/(2*1.225*Power.VTOL_A));

%Hover method ii
```

```

Power_req.hover_unittime_mii = 4*motor_fit(Aero.W_des*9.81/4);

%Hover method iii
RPM = VTOL_motor_fit1(Aero.W_des*9.81/4,0);
Power_req.hover_unittime_miii = 4*VTOL_motor_fit2(Aero.W_des*9.81/4,RPM)

Power_req = struct with fields:
    hover_unittime_mi: 6.8219e+03
    hover_unittime_mii: 1.4660e+04
    hover_unittime_miii: 8.1010e+03

```

```

%Selecting the actual prop team value
Power_req.hover_unittime = Power_req.hover_unittime_miii;

%Hover ceiling
power_max = 4*VTOL_motor_fit2(Aero.W_des*9.81/4,6000)

```

```

power_max = 1.1516e+04

```

```

power_max = 4*VTOL_motor_fit2(Aero.W_des*9.81/4,6000)

```

```

power_max = 1.1516e+04

```

```

sigma_maxalt = (Power_req.hover_unittime/power_max)^2

```

```

sigma_maxalt = 0.4949

```

```

maxalt.hover = alt_sigma(sigma_maxalt)

```

```

maxalt = struct with fields:
    conventional: 0
    hover: 6800

```

Transition optimisation

Climb Method 1 - Vertical Climb

```

%% METHOD A:
Vel.Vc = linspace(0,10,21); %Climb rate (assume constant climb)
Power_req.VTOL_climb_i = Power_req.hover_unittime_mi*(Vel.Vc./(2*Vel.Vh)+sqrt((Vel.Vc.,

%Estimation using equation of motion
VTOL.C_D = 1.17; %Model as a flat plate surface
VTOL.drag_climbrate = 0.5*1.225*Vel.Vc.^2*Power.VTOL_A*VTOL.C_D; %Assume that VTOL hapl

Thrust_req.VTOL_climb_ii = VTOL.drag_climbrate + Aero.W_des*9.81;
Power_req.VTOL_climb_ii = 4*motor_fit(Thrust_req.VTOL_climb_ii/4);
Power_req.VTOL_climb_ii = Power_req.VTOL_climb_ii';

RPM = VTOL_motor_fit1(Thrust_req.VTOL_climb_ii/4,Vel.Vc);
Power_req.VTOL_climb_iii = 4*VTOL_motor_fit2(Thrust_req.VTOL_climb_ii/4,RPM);

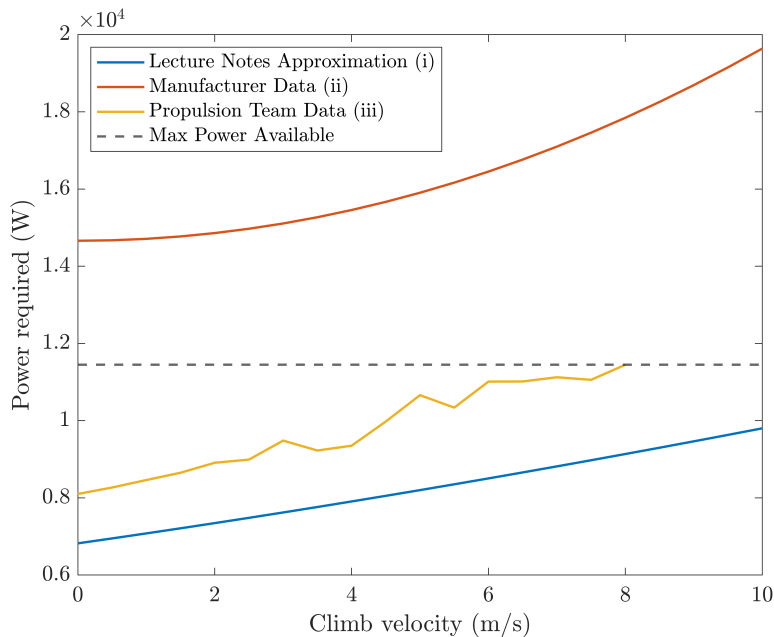
figure
plot(Vel.Vc,Power_req.VTOL_climb_i,'LineWidth',width)
hold on

```

```

plot(Vel.Vc,Power_req.VTOL_climb_ii,'LineWidth',width)
plot(Vel.Vc,Power_req.VTOL_climb_iii,'LineWidth',width)
yline(max(Power_req.VTOL_climb_iii),'--','LineWidth',width)
hold off
xlabel('Climb velocity (m/s)')
ylabel('Power required (W)')
legend('Lecture Notes Approximation (i)','Manufacturer Data (ii)','Propulsion Team Data (iii)')
set(gca,'fontsize', fontsize)
saveas(gcf,'VTOL.png')

```

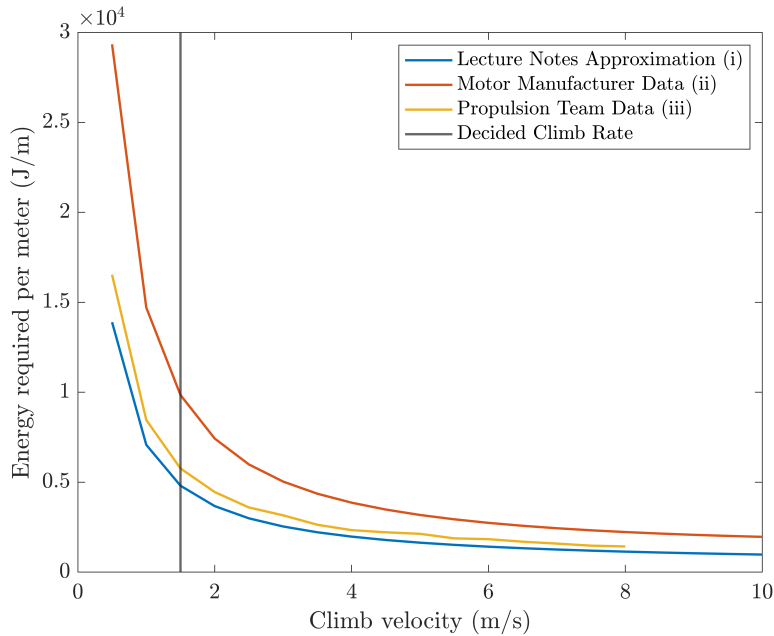


```

Energy_req.unitheight_m1i = Power_req.VTOL_climb_i.*(1./Vel.Vc); %Per meter
Energy_req.unitheight_m1ii = Power_req.VTOL_climb_ii.*(1./Vel.Vc); %Per meter
Energy_req.unitheight_m1iii = Power_req.VTOL_climb_iii.*(1./Vel.Vc); %Per meter

figure
plot(Vel.Vc,Energy_req.unitheight_m1i,'LineWidth',width)
hold on
plot(Vel.Vc,Energy_req.unitheight_m1ii,'LineWidth',width)
plot(Vel.Vc,Energy_req.unitheight_m1iii,'LineWidth',width)
xline(1.5,'LineWidth',width)
hold off
xlabel('Climb velocity (m/s)')
ylabel('Energy required per meter (J/m)')
legend('Lecture Notes Approximation (i)','Motor Manufacturer Data (ii)', 'Propulsion Team Data (iii)')
set(gca,'fontsize', fontsize)
saveas(gcf,'VTOLenergy.png')

```



```
%Choosing a climbrate to move forward with as the optimum
climbrate.VTOL = 1.5;

Energy_req.unitheight_m1i = Energy_req.unitheight_m1i(4);
Energy_req.unitheight_m1ii = Energy_req.unitheight_m1ii(4);
Energy_req.unitheight_m1iii = Energy_req.unitheight_m1iii(4);

%Vortex Ring State
Fn = Aero.W_des*9.81;
W = sqrt((8*Fn)/(1.225*pi*Power.D^2)); %Severe loss of lift and control

Total_time.VTOL = Mission.MSA/climbrate.VTOL;
```

Climb Method 2

```
climbrate.Joint = [0.5:1:4.5];

for i = 1:length(climbrate.Joint)

    velocity(i,:) = linspace(climbrate.Joint(i),Vel.V_s,100); %Climb velocity assume a
    gamma = -asind(climbrate.Joint(i)./velocity(i,:));

    L = 0.5*1.225*velocity(i,:).^2.*C_L_func(gamma,Aero.a_w,Aero.i_w,Aero.ZLA).*Aero.S
    C_D = Aero.C_D0 + C_L_func(gamma,Aero.a_w,Aero.i_w,Aero.ZLA).^2/(Aero.AR * Aero.e);
    D = 0.5*1.225*Aero.Sref*velocity(i,:).^2.*C_D;
    D_v = 0.5*1.225*climbrate.Joint(i).^2.*VTOL.C_D.*Power.VTOL_A;
```

```

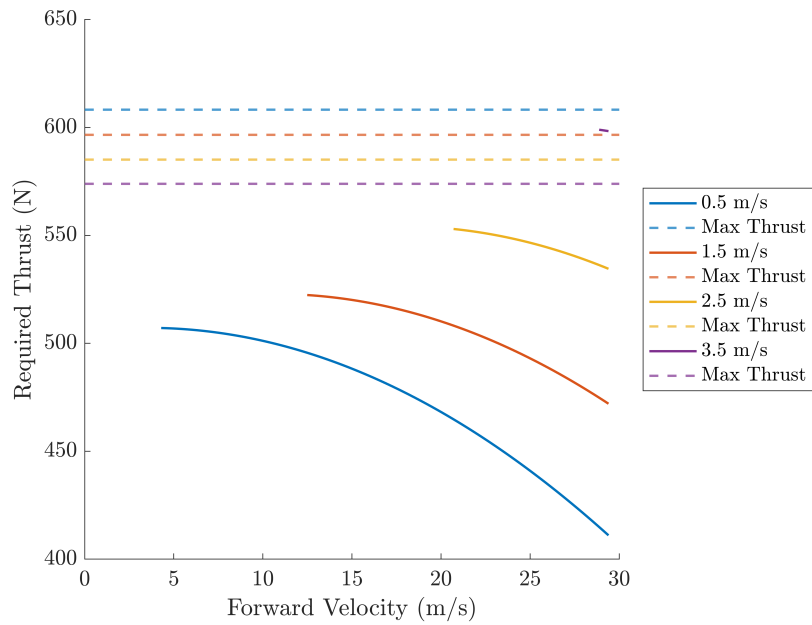
Thrust_req.vert_m2ii(i,:) = D.*sind(gamma) + D_v - L.*cosd(gamma) + Aero.W_des*9.8;

% Method ii
temp = 4*motor_fit(Thrust_req.vert_m2ii(i,:)./4);
Power_req.climb_m2ii(i,:) = temp.';
Energy_req.unitheight_m2ii(i,:) = Power_req.climb_m2ii(i,:)*(1/climbrate.Joint(i))

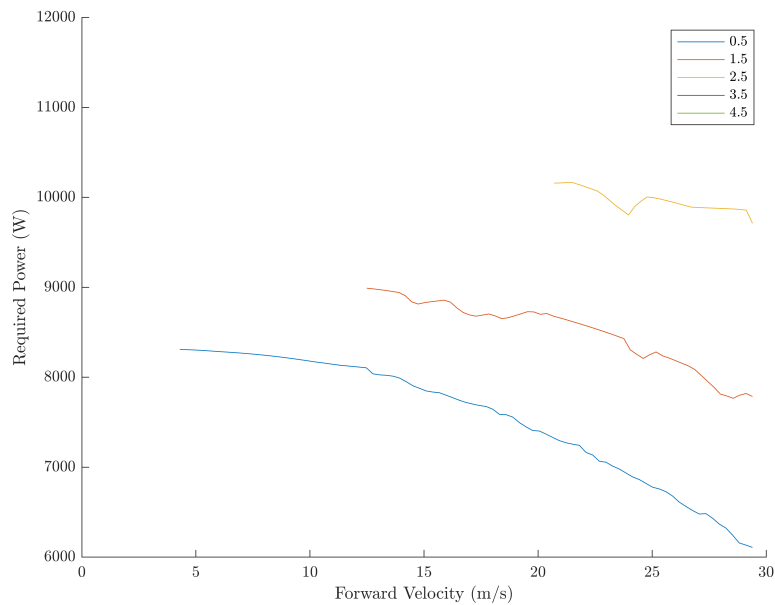
%Method iii
RPM = VTOL_motor_fit1(Thrust_req.vert_m2ii(i,:)./4,ones(1,length(Thrust_req.vert_m
Power_req.climb_m2iii(i,:) = 4*VTOL_motor_fit2(Thrust_req.vert_m2ii(i,:)/4,RPM);
Energy_req.unitheight_m2iii(i,:) = Power_req.climb_m2iii(i,:)*(1/climbrate.Joint(i
MAX_THRUST(i) = 4*max_thrust_v_climb(climbrate.Joint(i));
end

figure
hold on
plot(velocity(1,:),Thrust_req.vert_m2ii(1,:), 'Color',[0 0.4470 0.7410], 'LineWidth',wid
yline(MAX_THRUST(1), 'Color',[0 0.4470 0.7410], 'LineStyle','--', 'LineWidth',width)
plot(velocity(2,:),Thrust_req.vert_m2ii(2,:), 'Color',[0.85 0.325 0.098], 'LineWidth',wi
yline(MAX_THRUST(2), 'Color',[0.85 0.325 0.098], 'LineStyle','--', 'LineWidth',width)
plot(velocity(3,:),Thrust_req.vert_m2ii(3,:), 'Color',[0.9290 0.6940 0.125], 'LineWidth'
yline(MAX_THRUST(3), 'Color',[0.9290 0.6940 0.125], 'LineStyle','--', 'LineWidth',width)
plot(velocity(4,:),Thrust_req.vert_m2ii(4,:), 'Color',[0.4940 0.1840 0.5560], 'LineWidth
yline(MAX_THRUST(4), 'Color',[0.4940 0.1840 0.5560], 'LineStyle','--', 'LineWidth',width)
hold off
legend('0.5 m/s', 'Max Thrust', '1.5 m/s', 'Max Thrust', '2.5 m/s', 'Max Thrust', '3.5 m/s',
ylabel('Required Thrust (N)')
xlabel('Forward Velocity (m/s)')
set(gca, 'fontsize', fontsize)
saveas(gcf, 'Climb2Thrust.png')

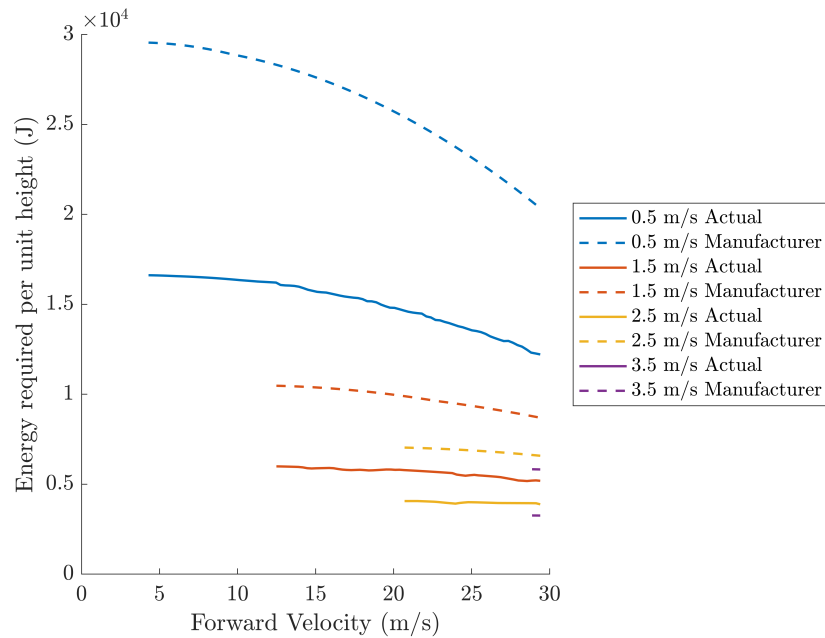
```



```
figure
hold on
plot(velocity(1,:),Power_req.climb_m2iii(1,:))
plot(velocity(2,:),Power_req.climb_m2iii(2,:))
plot(velocity(3,:),Power_req.climb_m2iii(3,:))
plot(velocity(4,:),Power_req.climb_m2iii(4,:))
plot(velocity(5,:),Power_req.climb_m2iii(5,:))
hold off
legend('0.5','1.5','2.5','3.5','4.5')
ylabel('Required Power (W)')
xlabel('Forward Velocity (m/s)')
```



```
figure
hold on
plot(velocity(1,:),Energy_req.unitheight_m2iii(1,:), 'Color',[0 0.4470 0.7410], 'LineWid
plot(velocity(1,:),Energy_req.unitheight_m2ii(1,:), 'Color',[0 0.4470 0.7410], 'LineStyl
plot(velocity(2,:),Energy_req.unitheight_m2iii(2,:), 'Color',[0.85 0.325 0.098], 'LineWid
plot(velocity(2,:),Energy_req.unitheight_m2ii(2,:), 'Color',[0.85 0.325 0.098], 'LineStyl
plot(velocity(3,:),Energy_req.unitheight_m2iii(3,:), 'Color',[0.9290 0.6940 0.125], 'Line
plot(velocity(3,:),Energy_req.unitheight_m2ii(3,:), 'Color',[0.9290 0.6940 0.125], 'Line
plot(velocity(4,:),Energy_req.unitheight_m2iii(4,:), 'Color',[0.4940 0.1840 0.5560], 'Li
plot(velocity(4,:),Energy_req.unitheight_m2ii(4,:), 'Color',[0.4940 0.1840 0.5560], 'Line
hold off
legend('0.5 m/s Actual','0.5 m/s Manufacturer','1.5 m/s Actual','1.5 m/s Manufacturer'
xlabel('Forward Velocity (m/s)')
ylabel('Energy required per unit height (J)')
set(gca,'fontsize', fontsize)
saveas(gcf,'Climb2.png')
```



```

Energy_req.unitheight_m2ii = min(Energy_req.unitheight_m2ii(2,:));
Energy_req.unitheight_m2iii = min(Energy_req.unitheight_m2iii(2,:));

%Assuming that the mass of fuel change is negligible
%Ignoring altitude effects as low

Energy_req.unitheightVTOL_climb = Energy_req.unitheight_m1iii;
Energy_req.unitheight_climb = Energy_req.unitheight_m2iii;
Energy_req.MSA = Energy_req.unitheightVTOL_climb * Mission.MSA;

```

Transition Method A

```

%% TRANSITION METHOD A - complete freefall
%Stall speed
Vel.V_s = sqrt((2*Aero.W_des*9.81)/(1.225*Aero.Sref*Aero.C_Lmax)); %Assume it can operi

PU_loadfactor = linspace(1.1,3,100);

t = linspace(0,180,1801);
delta_t = t(2)-t(1);

for j = 1:length(PU_loadfactor)

    %Pull up manoeuvre load factor
    n = PU_loadfactor(j);

```

```

vel_vert = 0.01; %Initialising the vertical velocity
h_vert = 0;
i = 1; %Initialise counter

%Vstall for a loadcase of n = 3 with a safety factor of 1.1
vel_pu(j) = 1.1*sqrt((2*Aero.W_des*9.81*n)/(1.225*Aero.Sref*Aero.C_Lmax)); %Assume

while vel_vert(i)<vel_pu(j)
    %Thrust - function of velocity
    T = max_thrust_V_h(vel_vert(i),15); %Estimation for the time being - height is

    T(isnan(T)) = 0;

    %Drag - function of velocity
    C_D = Aero.C_D0 + (1/(pi*Aero.e*Aero.AR))*Aero.C_Lmax^2; %Assuming level flight
    D = 0.5*1.225*Aero.Sref*vel_vert(i)^2*C_D;

    vel_vert(i+1) = vel_vert(i) + ((T-D+Aero.W_des*g)/(Aero.W_des))*delta_t; %Assu
    h_vert(i+1) = vel_vert(i)*delta_t + 0.5*((T-D+Aero.W_des*g)/(Aero.W_des))*delta_t;

    if vel_vert(i+1)==vel_vert(i)

        disp('Not possible for')
        disp(n)
        flag = 1;
        break

    else
        flag = 0;
        i = i+1; %Counter
    end

end

if flag == 1
    break
end

%Height required to free fall to V_stall for set load factor
h_vertical(j) = sum(h_vert);

%Height required to pull up for set load factor
h_pullup(j) = vel_pu(j)^2/(g*(n-1)); %Radius

%Total height required
h_total(j) = h_pullup(j) + h_vertical(j);

end

[h_vert,index] = min(h_total);
h_req.A = h_vert

h_req = struct with fields:
    A: 329.8093

```

```
n_opt = PU_loadfactor(index)
```

```
n_opt = 2.3091
```

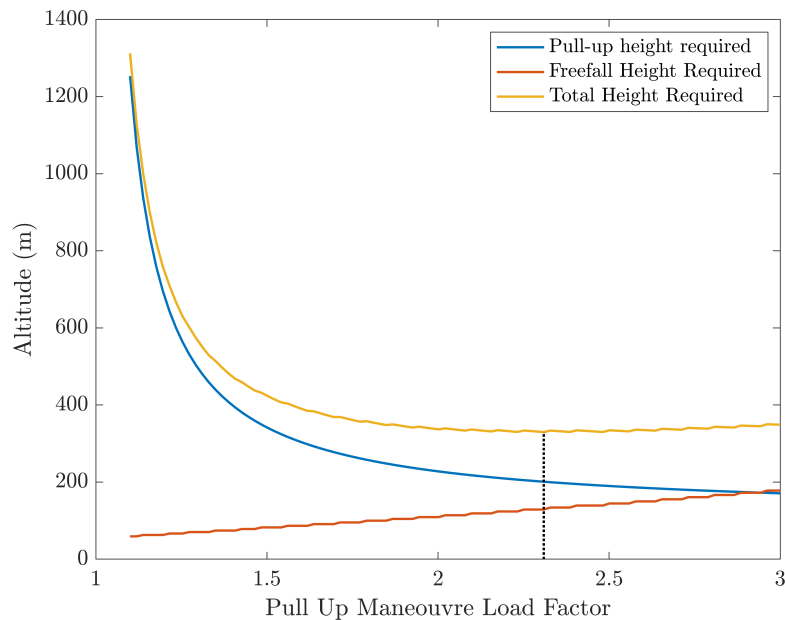
```
s.A = h_pullup(index);
```

```
figure
plot(PU_loadfactor(1:length(h_total)),h_pullup,'LineWidth',width)
hold on
plot(PU_loadfactor(1:length(h_total)),h_vertical,'LineWidth',width)
plot(PU_loadfactor(1:length(h_total)),h_total,'LineWidth',width)
temp1 = line([n_opt,n_opt],[0,h_vert])
```

```
temp1 =
  Line with properties:
      Color: [0 0.4470 0.7410]
  LineStyle: '-'
  LineWidth: 0.5000
    Marker: 'none'
  MarkerSize: 6
  MarkerFaceColor: 'none'
      XData: [2.3091 2.3091]
      YData: [0 329.8093]
      ZData: [1x0 double]
```

Show all properties

```
temp1.LineStyle = ':';
temp1.Color = 'k';
temp1.LineWidth = width;
hold off
xlabel('Pull Up Maneuvre Load Factor')
ylabel('Altitude (m)')
legend('Pull-up height required','Freefall Height Required','Total Height Required')
set(gca,'fontsize', fontsize)
saveas(gcf,'n_opt.png')
```



```
Energy_req.A = Energy_req.unitheight_climb*h_req.A;
```

Transition Method B

```
descent_angle = [1:0.2:Aero.stall_angle]; %stall angle as limiting factor

energy_B_ii = 0;
energy_B_iii = 0;

h_final_B = 0;
h_hor_final_B = 0;

for j=1:length(descent_angle)

    da = descent_angle(j);

    %Resetting / Initialising Quantities
    vel_hor = 0.01;
    vel_vert = 0.01;
    vel_inf = 0.01;

    accel_hor = 0; %Resetting acceleration
    accel_vert = 0; %Initialising vertical acceleration

    h_hor = 0;
    h_vert = 0;
```

```

Lift_B = 0;

i = 1; %Initialise counter

req_thrust_vert = Aero.W_des*9.81; %Initialising the required thrust (hover)
RPM = VTOL_motor_fit1(req_thrust_vert/4,vel_vert(i));
power_req_B_iii = 4*VTOL_motor_fit2(req_thrust_vert/4,RPM);

C_L = C_L_func(da,Aero.a_w,Aero.i_w,Aero.ZLA);

V_s = sqrt((2*Aero.W_des*9.81)/(1.225*Aero.Sref*C_L)); %Assuming at sea level

while vel_inf(i)<V_s

    %Thrust - function of velocity
    T(i) = max_thrust_V_h(vel_hor(i),Mission.MSA); %Estimation for the time being
    T(isnan(T)) = 0;

    %Forward Flight Drag - function of velocity
    C_D = Aero.C_D0 + (1/(pi*Aero.e*Aero.AR))*C_L^2; %Assuming level flight
    D = 0.5*1.225*Aero.Sref*vel_hor(i)^2*C_D;

    accel_hor = (T(i) - D*cosd(da) + Lift_B(i)*sind(da))/Aero.W_des;
    vel_hor(i+1) = vel_hor(i) + accel_hor*delta_t; %Assuming constant acceleration
    h_hor(i+1) = h_hor(i) + (vel_hor(i+1)+vel_hor(i))*delta_t/2;

    if vel_hor(i+1)-vel_hor(i)<tol
        disp('Not possible for AoA for')
        disp(da)
        h_hor_final_B(j) = nan;
        energy_B(j) = nan;
        hfinal_B(j) = nan;
        flags = 0;
        break
    end

    h_vert(i+1) = h_hor(i+1)*tand(da); %Cumulative sum of height
    vel_vert(i+1) = (h_vert(i+1)-h_vert(i))/delta_t; %instantaneous velocity
    accel_vert = (vel_vert(i+1)-vel_vert(i))/delta_t; %instantaneous acceleration
    D_vert = 0.5*1.225*Power.VTOL_A*VTOL.C_D*vel_vert(i+1)^2;

    vel_inf(i+1) = sqrt(vel_vert(i+1)^2 + vel_hor(i+1)^2);

    Lift_B(i+1) = 0.5*1.225*Aero.Sref.*C_L.*(vel_inf(i+1).^2);

    req_thrust_vert(i+1) = Aero.W_des*9.81 + Aero.W_des*accel_vert - Lift_B(i+1)*cos(da);
    RPM(i+1) = VTOL_motor_fit1(req_thrust_vert(i+1)/4,vel_vert(i+1));
    power_req_B_iii(i+1) = 4*VTOL_motor_fit2(req_thrust_vert(i+1)/4,RPM(i+1));

    i = i+1;

end

```

```

%Method ii
power_req_B_ii = 4*motor_fit(req_thrust_vert/4);
energy_B_ii(j) = trapz(delta_t,power_req_B_ii);

%Method iii
power_req_B_iii = rmmissing(power_req_B_iii);
energy_B_iii(j) = trapz(delta_t,power_req_B_iii);

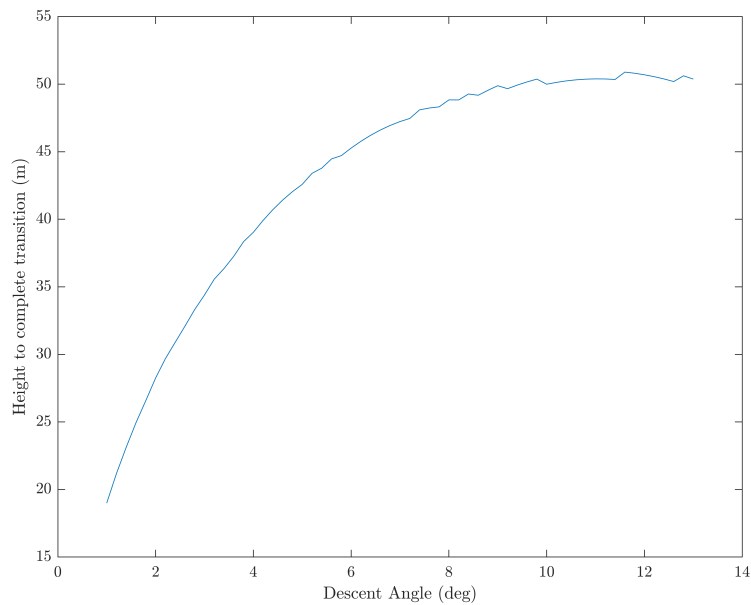
h_final_B(j) = h_vert(end);
h_hor_final_B(j) = h_hor(end);

end

[Energy_req.B_ii,index] = min(energy_B_ii + Energy_req.unitheight_climb*h_final_B);
[Energy_req.B_iii,index] = min(energy_B_iii + Energy_req.unitheight_climb*h_final_B);
h_req.B = h_final_B(index);
s.B = h_hor_final_B(index);

figure
plot(descent_angle,h_final_B)
xlabel('Descent Angle (deg)')
ylabel('Height to complete transition (m)')

```



```

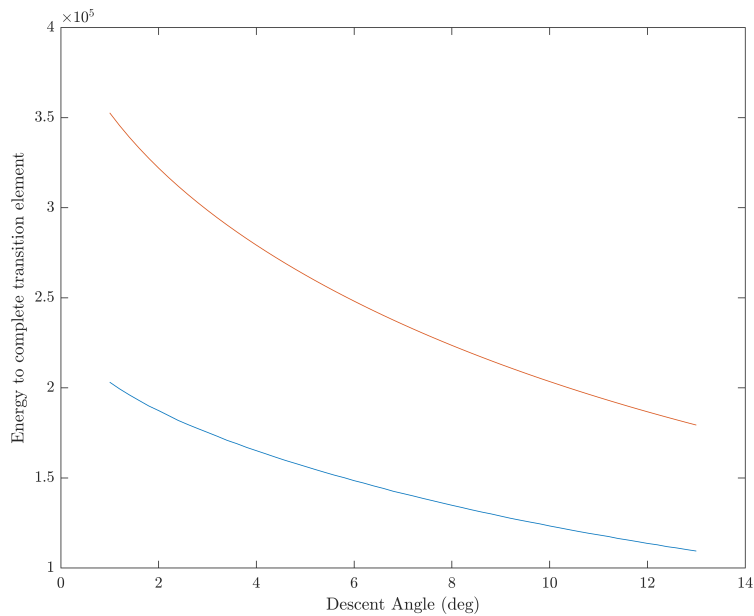
figure
plot(descent_angle,energy_B_iii)
hold on
plot(descent_angle,energy_B_ii)
hold off

```

```

xlabel('Descent Angle (deg)')
ylabel('Energy to complete transition element')

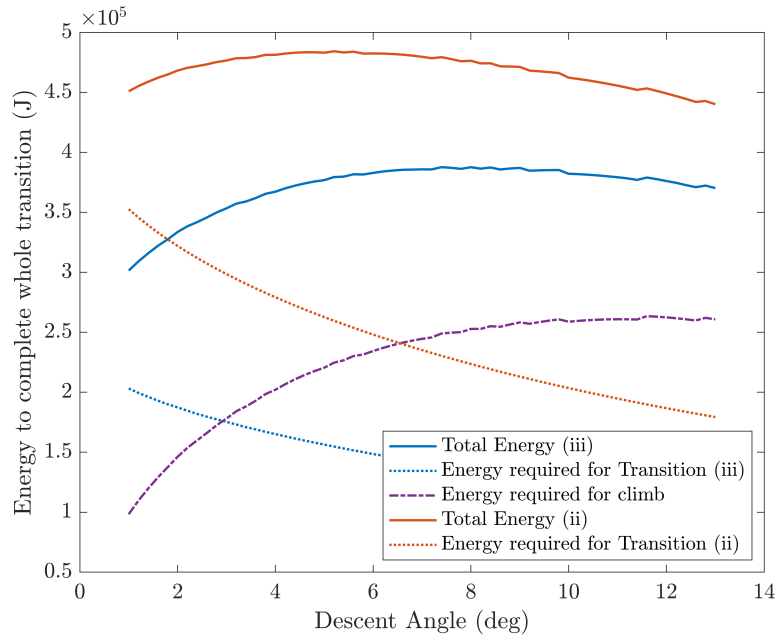
```



```

figure
plot(descent_angle,energy_B_iii + Energy_req.unitheight_climb*h_final_B,'Color',[0 0.4470 0.7410],\
hold on
plot(descent_angle,energy_B_iii,'Color',[0 0.4470 0.7410],'LineStyle',':','LineWidth',1)
plot(descent_angle,Energy_req.unitheight_climb*h_final_B,'Color',[0.4940 0.1840 0.5560],\
plot(descent_angle,energy_B_ii + Energy_req.unitheight_climb*h_final_B,'Color',[0.85 0.325 0.098],\
plot(descent_angle,energy_B_ii,'Color',[0.85 0.325 0.098],'LineStyle',':','LineWidth',1)
hold off
xlabel('Descent Angle (deg)')
ylabel('Energy to complete whole transition (J)')
legend('Total Energy (iii)','Energy required for Transition (iii)','Energy required for Transition (ii)')
set(gca,'fontsize', fontsize)
saveas(gcf,'TransB.png')

```



```
Energy_req.B = Energy_req.B_iii;
Energy_req.B0 = energy_B_iii(1);
```

Transition Method C

Descent aligned with freestream

```
%Descending inclined to the same angle of descent
descent_angle = [1:1:21]; %degrees (21 DEG MAX FOR CONTROLLABILITY)

h_final_C = 0;
h_hor_final_C = 0;

energy_C = 0;

for j=1:length(descent_angle)

    da = descent_angle(j);

    %Resetting / Initialising Quantities
    vel_hor = 0.01; %Initialising the horizontal velocity
    vel_vert = 0.01;
    vel_inf = 0.01;

    accel_hor = 0; %Resetting acceleration
    accel_vert = 0; %Initialising vertical acceleration
```

```

h_hor = 0;
h_vert = 0;

Lift_C = 0;

i = 1; %Initialise counter

req_thrust_vert = Aero.W_des*9.81/cosd(da); %Initialising the required thrust (hover)
C_L = C_L_func(0,Aero.a_w,Aero.i_w,Aero.ZLA);
V_s = sqrt((2*Aero.W_des*9.81)/(1.225*Aero.Sref*C_L)); %Assuming at sea level
while vel_inf(i)<V_s

    %Thrust - function of velocity
    T = max_thrust_V_h(vel_inf(i),Mission.MSA); %Estimation for the time being
    T(isnan(T)) = 0;

    %Forward Flight Drag - function of velocity
    C_D = Aero.C_D0 + (1/(pi*Aero.e*Aero.AR))*C_L^2; %Assuming level flight
    D = 0.5*1.225*Aero.Sref*vel_inf(i)^2*C_D;

    accel_hor = (T*cosd(da) + Lift_C(i)*sind(da) + req_thrust_vert(i)*sind(da) - D)/Aero.W_des;
    vel_hor(i+1) = vel_hor(i) + accel_hor*delta_t; %Assuming constant acceleration
    h_hor(i+1) = h_hor(i) + (vel_hor(i)+vel_hor(i+1))*delta_t/2;

    h_vert(i+1) = h_hor(i+1)*tand(da); %Cumulative sum of height
    vel_vert(i+1) = (h_vert(i+1)-h_vert(i))/delta_t; %instantaneous velocity
    accel_vert = (vel_vert(i+1)-vel_vert(i))/delta_t; %instantaneous acceleration

    vel_inf(i+1) = sqrt(vel_vert(i+1)^2 + vel_hor(i+1)^2);

    Lift_C(i+1) = 0.5*1.225*Aero.Sref.*C_L.*(vel_inf(i+1).^2);

    req_thrust_vert(i+1) = (Aero.W_des*accel_vert + T*sind(da) + Aero.W_des*9.81 -

if vel_inf(i+1) - vel_inf(i) < tol
    disp('Not possible for angle')
    disp(da)
    req_thrust_vert = nan;
    energy_Cii(j) = nan;
    energy_Ciii(j) = nan;
    power_req_C_ii = nan;
    power_req_C_iii = nan;
    h_final_C(j) = nan;
    h_hor_final_C(j) = nan;
    break
else
    %Method ii
    power_req_C_ii = 4*motor_fit(req_thrust_vert/4);
    energy_C_ii(j) = trapz(delta_t,power_req_C_ii);

    %Method iii

```

```

        RPM = VTOL_motor_fit1(req_thrust_vert/4,vel_vert);
        power_req_C_iii = 4*VTOL_motor_fit2(req_thrust_vert/4,RPM);
        power_req_C_iii = rmmissing(power_req_C_iii);
        energy_C_iii(j) = trapz(delta_t,power_req_C_iii);

        h_final_C(j) = h_vert(end);
        h_hor_final_C(j) = h_hor(end);

        i = i+1;
    end

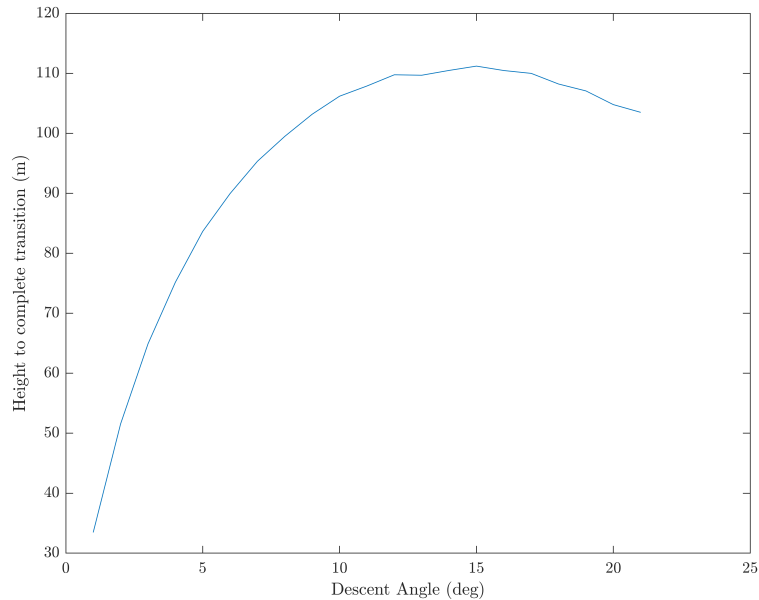
end

end

[Energy_req.C_ii,index] = min(energy_C_ii + Energy_req.unitheight_climb.*h_final_C);
[Energy_req.C_iii,index] = min(energy_C_iii + Energy_req.unitheight_climb.*h_final_C);
h_req.C = h_final_C(index);
Energy_req.C = Energy_req.C_iii;
Energy_req.C0 = energy_C_iii(1);
s.C = h_hor_final_C(index);

figure
plot(descent_angle,h_final_C)
xlabel('Descent Angle (deg)')
ylabel('Height to complete transition (m)')

```

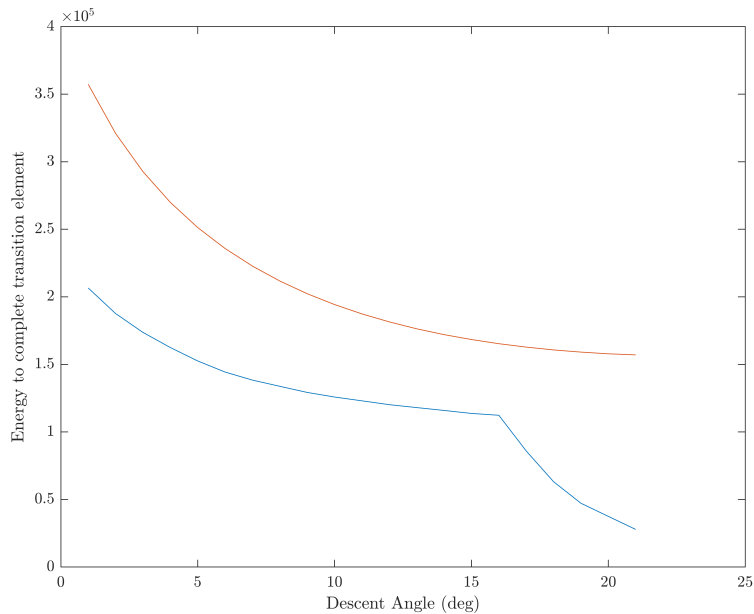


figure

```

plot(descent_angle,energy_C_iii)
hold on
plot(descent_angle,energy_C_ii)
hold off
xlabel('Descent Angle (deg)')
ylabel('Energy to complete transition element')

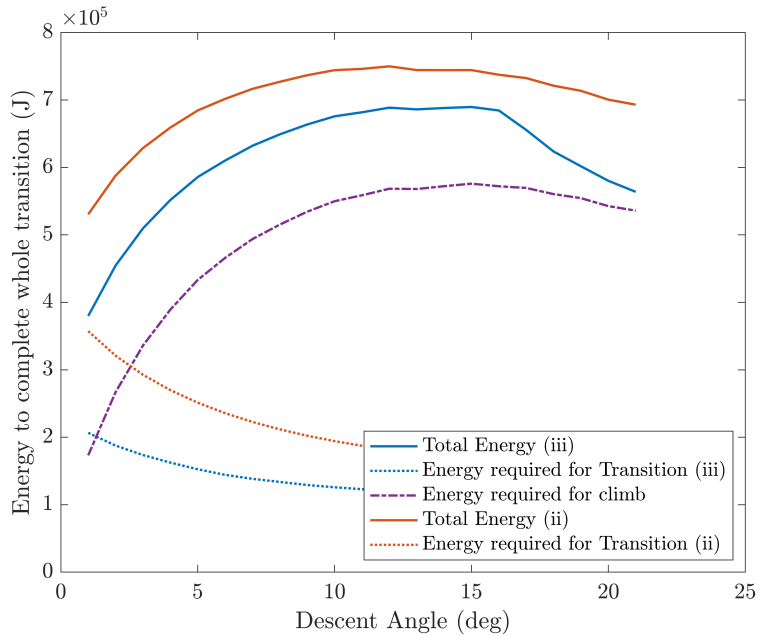
```



```

figure
plot(descent_angle,energy_C_iii + Energy_req.unitheight_climb*h_final_C, 'Color',[0 0.4 0.4], 'LineStyle','-', 'LineWidth',1)
hold on
plot(descent_angle,energy_C_iii, 'Color',[0 0.4470 0.7410], 'LineStyle',':', 'LineWidth',1)
plot(descent_angle,Energy_req.unitheight_climb*h_final_C, 'Color',[0.4940 0.1840 0.5560], 'LineStyle','-', 'LineWidth',1)
plot(descent_angle,energy_C_ii + Energy_req.unitheight_climb*h_final_C, 'Color',[0.85 0 0.85], 'LineStyle','-', 'LineWidth',1)
plot(descent_angle,energy_C_ii, 'Color',[0.85 0.325 0.098], 'LineStyle',':', 'LineWidth',1)
hold off
xlabel('Descent Angle (deg)')
ylabel('Energy to complete whole transition (J)')
legend('Total Energy (iii)', 'Energy required for Transition (iii)', 'Energy required for Transition (ii)', 'Energy required for Transition (i)')
set(gca, 'fontsize', fontsize)
saveas(gcf, 'TransC.png')

```



Transition Method D

Level Acceleration

```
C_L = C_L_func(0.2,Aero.a_w,Aero.i_w,Aero.ZLA)
```

```
C_L = 0.3469
```

```
Vel.V_szero = sqrt((2*Aero.W_des*9.81)/(Aero.Sref*C_L*1.225));
vel_hor = 0.01; %Initialising the horizontal velocity
Lift = 0; %Initialise Lift
i = 1; %Initialise counter
h_hor = 0;
accel_hor = 0; %Resetting acceleration
req_thrust_vert = Aero.W_des*9.81; %Initialising the required thrust
while vel_hor(i)<Vel.V_szero
    %Thrust - function of velocity
    T(i) = max_thrust_V_h(vel_hor(i),Mission.MSA); %Estimation for the time being
    T(isnan(T)) = 0;
```

```

%Drag - function of velocity
C_D = Aero.C_D0 + (1/(pi*Aero.e*Aero.AR))*C_L^2; %Assuming level flight
D = 0.5*1.225*Aero.Sref*vel_hor(i)^2*C_D;

accel_hor = (T(i)-D)/Aero.W_des;
vel_hor(i+1) = vel_hor(i) + accel_hor*delta_t; %Assuming constant acceleration betw
h_hor(i+1) = h_hor(i) + (vel_hor(i)+vel_hor(i+1))*delta_t/2;
Lift_D(i+1) = 0.5*1.225*vel_hor(i+1)^2*Aero.Sref*C_L;
req_thrust_vert(i+1) = Aero.W_des*9.81 - Lift_D(i+1); %Thrust required for level f

if vel_hor(i+1) - vel_hor(i)<tol
    disp('Not possible!')
    flag = 1;
    break
else
    i = i+1;
    flag = 0;
end

end

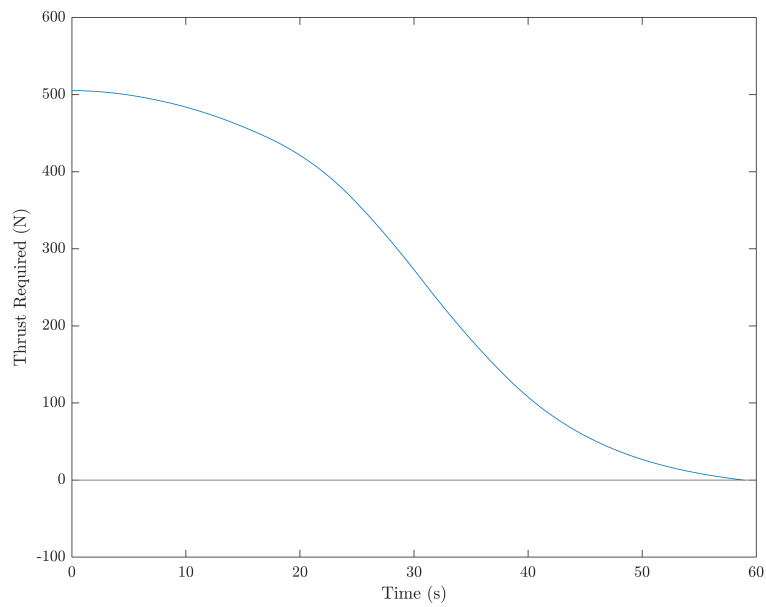
s.D = h_hor(end);

if flag == 0
    Power_req.D_ii = 4*motor_fit(req_thrust_vert/4); %Watt per motor
    Energy_req.D_ii = trapz(delta_t,Power_req.D_ii);

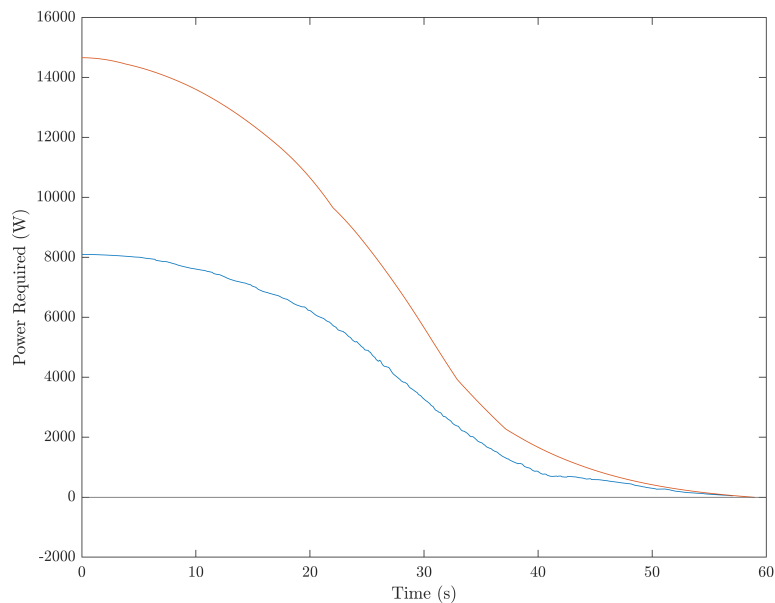
    RPM = VTOL_motor_fit1(req_thrust_vert./4,zeros(1,length(req_thrust_vert)));
    Power_req.D_iii = 4*VTOL_motor_fit2(req_thrust_vert./4,RPM);
    Power_req.D_iii = rmmissing(Power_req.D_iii);
    Energy_req.D_iii = trapz(delta_t,Power_req.D_iii);
    Total_time.Transition = t(i);
else
    Power_req.D_iii = nan; %Watt per motor
    Energy_req.D_iii = nan;
    Power_req.D_ii = nan; %Watt per motor
    Energy_req.D_ii = nan;
    Total_time.Transition = nan;
end

figure
plot(t(1:length(req_thrust_vert)),req_thrust_vert)
hold on
yline(0)
hold off
xlabel('Time (s)')
ylabel('Thrust Required (N)')

```



```
figure
plot(t(1:length(Power_req.D_iii)),Power_req.D_iii)
hold on
yline(0)
plot(t(1:length(Power_req.D_ii)),Power_req.D_ii)
hold off
xlabel('Time (s)')
ylabel('Power Required (W)')
saveas(gcf,'TransD_power.png')
```



Optimum cruise height

```
% Optimising time and energy usage - prioritise time (lqr style)
%climb - not taking TAS into account

heights = [350:50:3000]; %100m to 3000m
[~,~,~,rho] = atmosisa(heights);
sigma = rho/1.225;

SEP.levels(2:6) = SEP.levels;
SEP.levels(1) = climbrate.Fixed_wing;

heightmach = fit(testheight',testmach','poly2');
heightclimbrate = fit(testheight',SEP.levels','poly2');

gamma = asind(climbrate.Fixed_wing/Vel.V_cl);

%Climb - best rate of climb at each altitude
for i = 1:length(heights)
    [time.climb(i),distance.climb(i),time.descent(i),distance.descent(i)] = climb_func
end

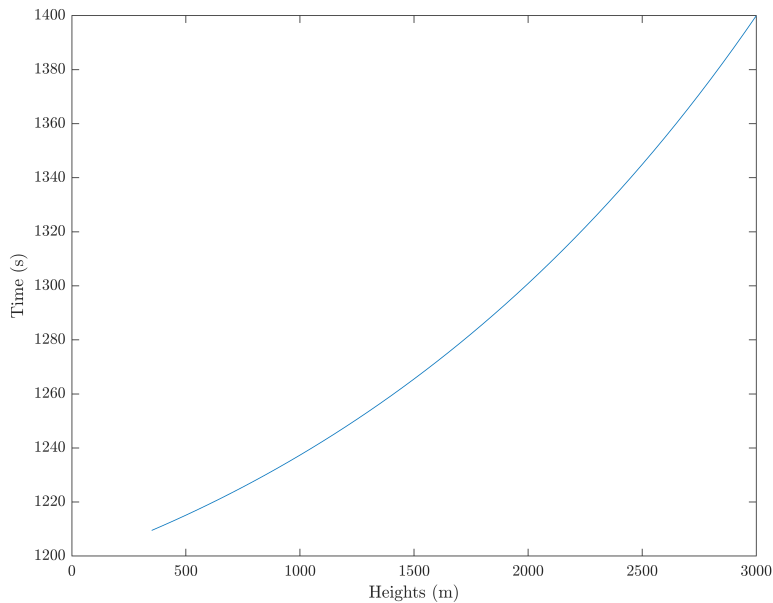
%Cruise segment
TAS.cruise = Vel.V_cruise./sqrt(sigma);
distance.cruise = 70000 - distance.climb - distance.descent;
time.cruise = distance.cruise./TAS.cruise;
```

```

time.total_cruise = time.descent + time.climb + time.cruise;

figure
plot(heights,time.total_cruise)
xlabel('Heights (m)')
ylabel('Time (s)')

```



```

%Selecting values
time.descent = time.descent(1);
time.climb = time.climb(1);
time.cruise = time.cruise(1);

time.total_cruise = time.descent + time.climb + time.cruise;

distance.descent = distance.descent(1);
distance.climb = distance.climb(1);
distance.cruise = distance.cruise(1);

[~,index] = min(time.total_cruise);
opt_height = heights(index)

opt_height = 350

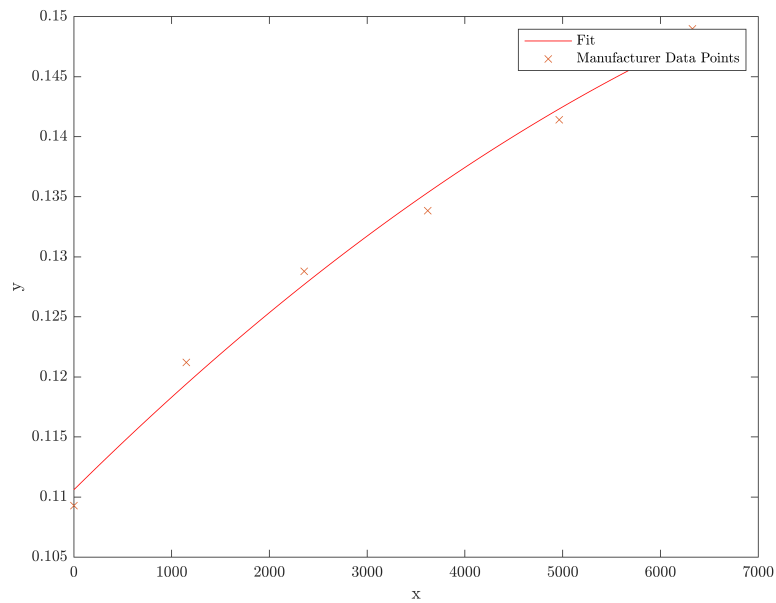
```

```

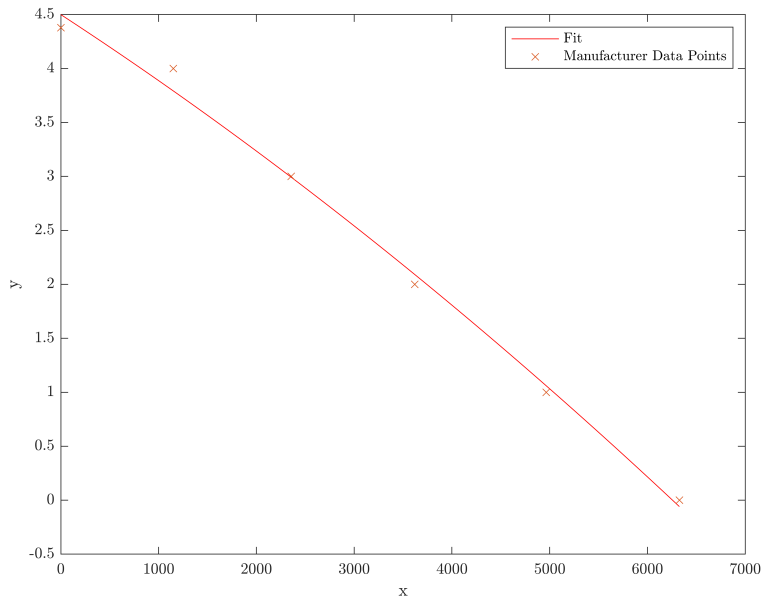
figure
plot(heightmach)
hold on
plot(testheight,testmach,'x')

```

```
hold off
legend('Fit','Manufacturer Data Points')
```



```
figure
plot(heightclimbrate)
hold on
plot(testheight,SEP.levels,'x')
hold off
legend('Fit','Manufacturer Data Points')
```



Fixed wing to VTOL Transition

```

vel_hor = Vel.V_szero; %Initialising the horizontal velocity

Lift = 0; %Initialise Lift

C_D0_off = 1.17;

i = 1; %Initialise counter

accel_hor = 0; %Resetting acceleration

req_thrust_vert = 0; %Initialising the required thrust

Aero.C_L0 = C_L_func(0,Aero.a_w,Aero.i_w,Aero.ZLA);

while vel_hor(i)>10

    %Drag - function of velocity
    C_D = Aero.C_D0 + (1/(pi*Aero.e*Aero.AR))*Aero.C_L0^2; %Assuming level flight mode
    D = 0.5*1.225*vel_hor(i)^2*(C_D0_off*Power.ICE_A+C_D*Aero.Sref);

    vel_hor(i+1) = vel_hor(i) + ((-D)/(Aero.W_des))*delta_t; %Assuming constant accele
    h_hor(i+1) = h_hor(i)+vel_hor(i)*delta_t + 0.5*((-D)/(Aero.W_des))*delta_t^2;
    Lift(i+1) = 0.5*1.225*vel_hor(i+1)^2*Aero.Sref*Aero.C_L0;
    req_thrust_vert(i+1) = Aero.W_des*9.81 - Lift(i+1); %Thrust required for level flight

```

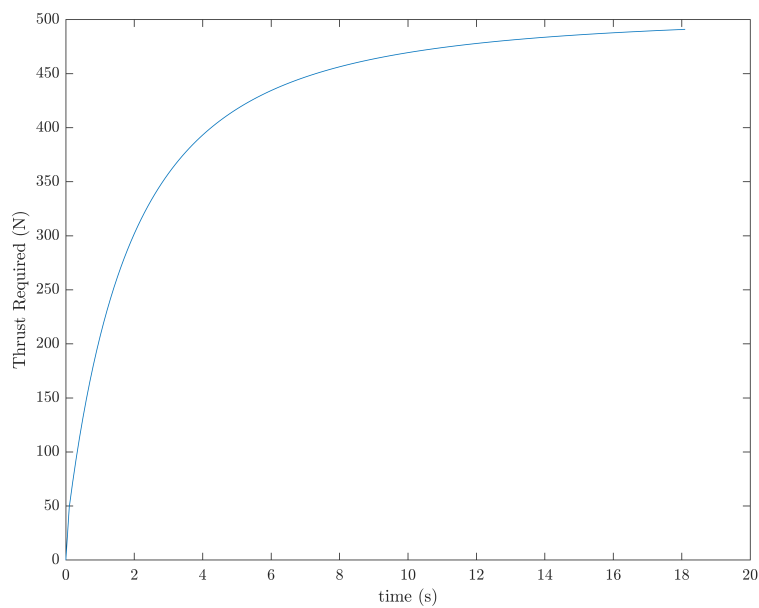
```

    i = i+1;

end

figure
plot(t(1:length(req_thrust_vert)),req_thrust_vert)
xlabel('time (s)')
ylabel('Thrust Required (N)')

```



```

Power_req.altTrans = 4*motor_fit(req_thrust_vert/4);
Energy_req.altTrans = trapz(t(1:length(Power_req.altTrans)),Power_req.altTrans);
Total_time.altTrans = t(i);
s.altTrans = h_hor(end);
time.altTrans = t(i);

```

VTOL Descent

```

%Assuming a steady descent
climbrate.VTOLdescent1 = -2; %MSA - 5m
climbrate.VTOLdescent2 = -0.5; %5-0m

h_req.VTOLdescent1 = Mission.MSA-5; %MSA to 5m
h_req.VTOLdescent2 = 5; %5m to 0m

D = 0.5*1.225*Power.VTOL_A*climbrate.VTOLdescent1^2*VTOL.C_D;
req_thrust1 = Aero.W_des*9.81 - D;

```

```

energy1_ii = 4*motor_fit(req_thrust1/4)*(h_req.VTOLdescent1/climbrate.VTOLdescent1);
RPM = VTOL_motor_fit1(req_thrust1/4,climbrate.VTOLdescent1);
energy1_iii = 4*VTOL_motor_fit2(req_thrust1/4,RPM) * (h_req.VTOLdescent1/climbrate.VTOLdescent1);

D = 0.5*1.225*Power.VTOL_A*climbrate.VTOLdescent2^2*VTOL.C_D;
req_thrust2 = Aero.W_des*9.81 - D;
energy2_ii = 4*motor_fit(req_thrust2/4)*(h_req.VTOLdescent2/climbrate.VTOLdescent2);
RPM = VTOL_motor_fit1(req_thrust2/4,climbrate.VTOLdescent2);
energy2_iii = 4*VTOL_motor_fit2(req_thrust2/4,RPM) * (h_req.VTOLdescent2/climbrate.VTOLdescent2);

Energy_req.VTOL_Descent = energy1_iii + energy2_iii;

```

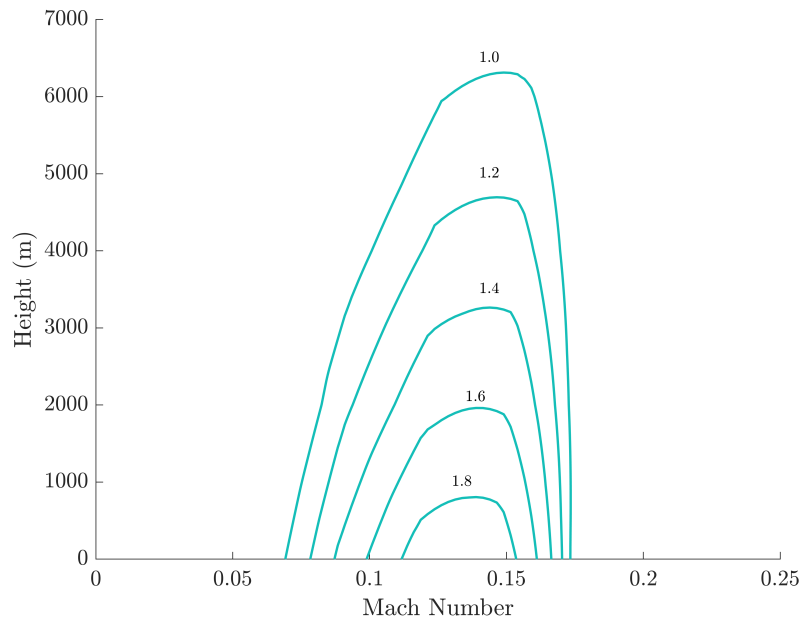
Turn Performance

```

n = [1:0.2:2];
Aero.C_Lloiter = 0.7;

figure
hold on
for i = 1:length(n)
    SEP.DRAG = Drag_func(SEP.VELOCITY,SEP.SIGMA,Aero.C_D0,Aero.W_des,Aero.AR,Aero.e,Aero.Ce)
    SEP.Ps = SEP.VELOCITY.*((SEP.THRUST/(Aero.W_des*g))-((SEP.DRAG)./(Aero.W_des*9.81)))
    [SEP.C,SEP.h] = contour(SEP.MACH,SEP.HEIGHT,SEP.Ps,[0 0],'LineWidth',width);
end
text(0.13,1000,'1.8')
text(0.135,2100,'1.6')
text(0.14,3500,'1.4')
text(0.14,5000,'1.2')
text(0.14,6500,'1.0')
hold off
xlabel('Mach Number','Interpreter','latex')
ylabel('Height (m)','Interpreter','latex')
set(gca,'TickLabelInterpreter','latex')
set(gca,'fontsize', fontsize)
saveas(gcf,'turn.png')

```



```
Vel.turn = sqrt((2*Aero.W_des*9.81*n)./(1.225*Aero.Sref*Aero.C_Lloiter));
R = (Vel.turn.^2)./(g.*sqrt(n.^2 - 1));
```

Final Performance Estimates

```
C.cruise1_final = 1.763/3600; %kg per second
C.loiter_final = 0.564/3600;
C.cruise2_final = 1.258/3600;

C.cruise1_final = 0.590*7.200/3600; %kg/sec
C.loiter_final = 0.590*3.300/3600;
C.cruise2_final = 0.590*4.700/3600;

%VTOL climb 15m
time.VTOL1 = Mission.MSA/climbrate.VTOL;
s.VTOL = 0; %0 lateral movement
Battery_req.MSA = (Energy_req.MSA/3600)/Power.energy_density; %3600 converts to watt h

%Transition to Level Flight (Trans 1)
s.trans1 = 0;
Battery_req.trans1 = (Energy_req.D_iii/3600)/Power.energy_density;
Fuel_req.trans1 = Total_time.Transition*C.cruise1_final;

%Climb to altitude (Climb 1)
Fuel_req.climb1 = time.climb*C.cruise1_final;
time.climb1 = time.climb;
```

```

s.climb1 = distance.climb;

%Descent (Descent 1)
Fuel_req.descent1 = 0;
time.descent1 = time.descent;
s.descent1 = distance.descent/1000; %V cruise or V-loiter for most efficient glide?

%Cruise 1
Fuel_req.cruise1 = time.cruise*C.cruise1_final;
time.cruise1 = time.cruise;
s.cruise1 = distance.cruise;

%Loiter
time.loiter = 8.7*60;
Fuel_req.loiter = time.loiter * C.loiter_final;

%Transition to VTOL (altTrans1)
s.altTrans1 = s.altTrans;
Battery_req.altTrans1 = (Energy_req.altTrans/3600)/Power.energy_density;

%Hover
Battery_req.Hover = (Power_req.hover_unittime*Mission.hover/3600)/Power.energy_density

%Transition to Level Flight (Trans 2)
Battery_req.Trans2 = (Energy_req.D_iii/3600)/Power.energy_density;
s.trans2 = s.D;
Fuel_req.trans2 = Fuel_req.trans1;

%Climb to altitude
Fuel_req.climb2 = 0;
time.climb2 = 0;
s.climb2 = 0;

%Cruise 2 - operated at L/D max speed
Fuel_req.cruise2 = ((70000-s.climb2-s.trans2)/Vel.V_cruise2)*C.cruise2_final;
time.cruise2 = (70000-s.climb1-s.trans1)/Vel.V_cruise;
s.cruise2 = 70000-s.trans1;

%Descent
Fuel_req.descent2 = 0;
time.descent2 = 0;
s.descent2 = 0;

%Transition to VTOL (Trans_VTOL 2)
s.altTrans2 = 0;
Battery_req.altTrans2 = (Energy_req.altTrans/3600)/Power.energy_density;

%VTOL descent 15m
% Battery_req.VTOL_Descent = (Energy_req.VTOL_Descent/3600)/Power.energy_density;
Battery_req.VTOL_Descent = (Power_req.hover_unittime*8/3600)/Power.energy_density;

%TOTAL REQUIRED
Battery_req.TOTAL = Battery_req.MSA + Battery_req.trans1 + Battery_req.Hover + Battery_req.VTOL_Descent;
Fuel_req.TOTAL = Fuel_req.trans1 + Fuel_req.climb1 + Fuel_req.cruise1 + Fuel_req.trans2 + Fuel_req.climb2 + Fuel_req.descent1 + Fuel_req.descent2 + Fuel_req.loiter + Fuel_req.cruise2 + Fuel_req.VTOL_Descent;

```

```

%EXCESS BATTERY
Battery_req.Excess = 5.5 - Battery_req.TOTAL;
additionalhover = Battery_req.Excess*Power.energy_density*3600/Power_req.hover_unittime;

additionalhover = 409.5115

```

```

function Drag = Drag_func(velocity,sigma,C_D0,W_des,AR,e,Sref,n)
    %height is in m

    rho = sigma.*1.225;
    C_L = (2.*W_des.*9.81*n)./(rho.*(velocity.^2).*Sref);
    Cd = C_D0 + (C_L.^2)./(pi.*AR.*e);
    Drag = (0.5.*rho.*Sref.*Cd.*(velocity.^2));

end

function [mach] = Stall_func(height,C_Lmax,W_des,Sref)
    [~,a,~,rho] = atmosisa(height);
    velocity = sqrt((2*W_des*9.81)./(rho.*C_Lmax.*Sref));
    mach = velocity./a;
end

function height = alt_sigma(sigma)
    h = [0:100:20000];
    [~,~,~,rho] = atmosisa(h);
    sigma_array = rho./1.225;
    [~, index] = min(abs(sigma_array-sigma));
    height = h(index);
end

function C_L = C_L_func(alpha,a_w,i_w,alpha0)
    % alpha is in degrees reference to the level quadcopter
    % a_w is in radians
    % i_w is in degrees

    alpha(alpha>14.5) = nan;
    alpha(alpha<-7) = nan;

    C_L = (a_w * (alpha + i_w - alpha0) * pi/180);
end

function [height,mach] = max_climb_func(Ps,levels,height_array,mach_array)

    for i = 1:length(levels)

        temp = (Ps>=levels(i)-0.01) & (Ps<=levels(i)+0.01);

        temp_height = max(temp,[],2);
        height_index = find(temp_height,1,'last');
        mach_index = find(temp(height_index,:),1);
    end
end

```

```

        height(i) = height_array(height_index);
        mach(i) = mach_array(mach_index);

    end

end

function [time_climb,distance_climb,time_descent,distance_descent] = climb_func(height

    h_climb = [0:50:height];
    h_descent = [target_descent:50:height];

    distance_climb = 0;
    distance_descent = 0;
    time_descent = 0;
    time_climb = 0;

    %Height must be in 50m intervals
    for i = 1:length(h_climb)-1
        [~,a,~,~] = atmosisa(h_climb(i));

        TAS_climb = heightmach(h_climb(i))*a;

        climbrate(i) = heightclimbrate(h_climb(i));

        time_climb(i) = 50/climbrate(i);

        vel_hor_climb = TAS_climb*cosd(gamma);

        distance_climb(i+1) = distance_climb(i) + vel_hor_climb*time_climb(i);
    end

    for i = 1:length(h_descent)-1

        [~,~,~,rho] = atmosisa(h_climb(i));

        IAS_descent = vel_descent;
        TAS_descent = IAS_descent/(rho/1.225);

        descentrate = 500/(3.28*60);

        time_descent(i) = 50/descentrate;

        gamma = asind(descentrate/TAS_descent);

        vel_hor_descent = TAS_descent*cosd(gamma);

        distance_descent(i+1) = distance_descent(i) + vel_hor_descent*time_descent(i);
    end

    distance_climb = distance_climb(end);
    distance_descent = distance_descent(end);
    time_climb = sum(time_climb);
    time_descent = sum(time_descent);

```

end